

RTL Design Sherpa

DDR4 / LPDDR4 Family Controller Hardware Architecture Specification 0.5

October 3, 2026

Table of Contents

1 Andesite Has Index.....9

2 Document Information.....10

2.1 Related documents.....10

2.2 Terminology.....11

2.3 Revision History.....11

2.4 How to read this book.....17

3 Purpose and Scope.....19

3.1 Purpose.....19

3.2 Why a delta specification.....19

3.3 What is in scope.....19

3.4 What is out of scope.....20

4 Document Conventions.....21

4.1 Block marking.....21

4.2 What this book does not restate.....21

4.3 Signal and parameter naming.....21

4.4 Timing values.....22

4.5 Citations.....22

5 Definitions and Acronyms.....23

5.1 Interfaces.....23

5.2 DRAM commands this document refers to.....23

5.3 Terms new to andesite.....24

5.4 Terms used with a specific meaning here.....25

6 Scope and Goals....26
6.1 What andesite is....26
6.2 Goals....26
6.3 Non-goals....26
6.4 What makes DDR4/LPDDR4 easier than it looks....27
6.5 What is genuinely harder....27
7 Block Diagram....29
7.1 The whole controller, marked against scoria....29
7.1.1 Figure 2.1: andesite block diagram, with inheritance marking....29
7.2 Reading the figure....29
7.3 What the figure deliberately omits....30
8 Module Hierarchy....31
8.1 Planned hierarchy....31
8.1.1 Top and macro tier....31
8.1.2 FUB tier, inherited unchanged....32
8.1.3 FUB tier, modified....32
8.1.4 FUB tier, new....34
8.2 Naming, and the package....34
9 The Target Design Point....36
9.1 Named as of v0.1....36
9.2 What this fixes....37
9.3 What is still open....38
10 What Is Inherited, and the Ten Areas That Change....39

10.1 The inheritance, stated once....39
10.2 The changes, in one table....39
10.3 The ten areas, one paragraph each....41
10.4 The full reuse table....42
10.5 Dormant: powerdown_ctrl and dfi_signal_pack....44
10.6 Deferred, with named conditions....45
11 Init and ZQ: RESET#, MR0-MR6, Gear-Down....46
11.1 The sequence is JEDEC's, verbatim....46
11.2 What the sequencer owns....48
11.3 ZQ calibration....48
12 Training: Write Leveling Inherited, Read Leveling New....50
12.1 The family rule: interfaces in hardware, searches in firmware....50
12.2 Write leveling: wrlvl_ifc, MODIFIED....50
12.3 Read leveling: rdlvl_ifc, NEW....51
12.4 LPDDR4 CA/WDQ training: ca_train_ifc, NEW....51
12.5 What training costs the surrounding blocks....52
13 Refresh: FGR and Controller-Directed Per-Bank....53
13.1 Marking: MODIFIED — inherited mechanism, FGR density on top....53
13.2 What scoria already has, and what andesite keeps....53
13.3 The DDR4 delta: FGR....53
13.4 The LPDDR4 delta: the controller names the bank....54
13.5 Requirements....54
13.6 Telemetry....55

14 ODT: Dynamic, with a New odt_ctrl....56
14.1 Why static ODT doesn't survive DDR4....56
14.2 What odt_ctrl owns....56
14.3 Requirements....57
15 The LPDDR4 Deltas, Chapter by Chapter....59
15.1 Where LPDDR4 differs....59
15.2 What LPDDR4 does *not* change....60
16 DFI v4.0 Master Interface, and the v3.1 Delta....61
16.1 The revision decision....61
16.2 What does NOT change....61
16.3 What changes and matters: the 3.1 → 4.0 delta....62
16.4 What andesite does NOT implement....64
16.5 The MAS seam....65
17 AXI4 Slave and APB CSR (both inherited)....66
17.1 The host side is unchanged in shape....66
17.2 The register block....66
17.3 The seam with the rest of the book....66
18 andesite_pkg, and Build-Time vs Runtime....67
18.1 The package....67
18.2 The rule: timings are runtime, geometry is build-time....67
18.3 New CSR groups....69
18.4 The duplication, recorded again....70
19 Verification Strategy, and the Open Questions....71

19.1 The target is the DFI boundary, in simulation only....71
19.2 The inherited-bring-tests argument....75
19.3 What must be verified, beyond the inherited suites....75
19.4 Status, 2026-10-04....76
19.5 Open questions....76
19.6 What would make this a 1.0....79

List of Figures

Figure 2.1: andesite block diagram, with inheritance marking.....29

List of Tables

Table 1: Table 0.1: Document information....10
Table 2: Table 0.2: Related documents....10
Table 3: Table 0.3: Revision history....11
Table 4: Table 1.1: Block markings....21
Table 5: Table 1.2: Commands, and which are new to this family member....23
Table 6: Table 1.3: The long/short timing pairs bank groups introduce....24
Table 7: Table 2.1: Top and macro tier....31
Table 8: Table 2.2: Modified FUBs — every row in this table is MODIFIED, and the marking column says so rather than leaving it to the section heading....32
Table 9: Table 2.3: New FUBs — every row is NEW....34
Table 10: Table 2.4: The target design point....36
Table 11: Table 2.5: Build-time parameters fixed by the design point....37
Table 12: Table 3.0: The ten delta areas, verbatim from the bootstrap spec....39
Table 13: Table 3.1: The complete reuse table — every scoria module with its andesite marking....42
Table 14: Table 3.2: The deferred list....45
Table 15: Table 3.3: DDR4 initialization requirements, ordered per JESD79-4....46
Table 16: Table 3.4: LPDDR4 initialization requirements, ordered per JESD209-4 47
Table 17: Table 3.5: rdlvl_ifc responsibilities....51
Table 18: Table 3.6: ca_train_ifc responsibilities....51
Table 19: Table 3.7: odt_ctrl responsibilities....56
Table 20: Table 4.1: The 3.1 → 4.0 delta — what scoria left unimplemented is andesite's work list....61
Table 21: Table 4.2: Deliberately unimplemented at the 4.0 boundary....64
Table 22: Table 5.0: The design-point geometry, byte-identical to Chapter 2.4....68
Table 23: Table 5.1: Build-time versus runtime....68
Table 24: Table 5.2: New CSR groups....69
Table 25: Table 6.1: The open questions, each with the condition that answers it 75

1 Andesite Has Index

Generated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

Table 0.1: Document information

Field	Value
Title	andesite DDR4/LPDDR4 Family Controller — Hardware Architecture Specification
Version	0.5
Date	2026-10-05
Status	Integration edition: the scheduler macro is rewired to the P1 init/policy interface and the training/DFI/AXI4 plane assemblies plus `andesite_core` exist; 248 DV tests green at the DFI 4.0 boundary. Open questions remain in Chapter 6
Scope	Controller architecture to the DFI v4.0 boundary, for DDR4 and LPDDR4
Not in scope	The PHY; board bring-up; the DDR5 features DFI v4.x also carries

2.1 Related documents

Table 0.2: Related documents

Document	Where	Relationship
andesite bootstrap design spec	`../../../../../../docs/superpowers/specs/2026-10-03-andesite-bootstrap-design.md`	**binding foundation** — the scope decisions and settled filing
scoria HAS	`../../../scoria-ddr3-lpddr3/docs/scoria_has/`	the architecture andesite is derived from; the reuse pool
scoria design requirements	`../../../scoria-ddr3-lpddr3/docs/design-requirements.md`	its delta-analysis method and D1-D3 decisions apply here

Document	Where	Relationship
family docs	`../../../../docs/`	shared-core design and doctrine — owned by no single controller (see Table 0.1 note)
JESD79-4	outside the repo, cold storage	DDR4 device standard
JESD209-4	outside the repo, cold storage	LPDDR4 device standard
DFI v4.0	operator research storage `/mnt/data/github/dfi-specs/` (on disk 2026-10-04); study completed 2026-10-04 as andesite TASK-005	the PHY boundary; clause citations are real DFI 4.0 section numbers; remaining `§TBC(TASK-005)` suffixes mark unconfirmed claims

2.2 Terminology

INHERITED / MODIFIED / NEW The marking every block in this document carries, relative to scoria. *Inherited* means the scoria block is used as-is, and its correctness argument transfers with it. *Modified* means a named, bounded change. *New* means no scoria counterpart exists.

nCK A DRAM clock cycle, the unit JESD79-4 states most command-spacing minimums in.

Maintenance traffic Commands the controller must issue that no host requested — refresh and ZQ calibration. It competes with demand traffic for the command bus, which makes it a scheduling problem and not only a sequencing one.

DFI 4.0 §TBC(TASK-005) Read “clause to be confirmed by andesite TASK-005 (DFI 4.0 spec acquisition and BFM study)”. The specification is now on disk and the study ran on 2026-10-04; remaining suffixes mark claims whose clause numbers could not be confirmed, not claims whose substance is unverified.

2.3 Revision History

Table 0.3: Revision history

Version	Date	Change
0.5	2026-10-05	TASK-016 integration pass: the scheduler macro

Version	Date	Change
		and the three remaining plane assemblies land, and `andesite_core` exists. The macro is rewired to the P1 init/policy interface (DDR4-only init sequencer, honest `cmd_ack` qualified on MRS/ZQCL accept, `zq_cal_start_o` exported as observability, dead `dfi_init_*` handshake / `t_rfc_wait_i` / `t_xpr_wait_i` / `mr_wr_o` ports retired); `dfi_cmd_path` (new) routes the widened `{ap,col,row,bg,bank,rank,op}` command stream through the P1 formatter with the DDR4 A10 auto-precharge case-split; `andesite_training_plane` assembles the carried wrlvl/rdlvl/ca_train interfaces with a maintenance command channel; `andesite_dfi_plane` assembles the CDC + cmd_path + write serializer/read aligner with DBI packed through the write and return CDC FIFOs; `andesite_axi4_plane` is

Version	Date	Change
		born as the scoria rename-carry under a byte-parity gate and a logic-parity suite; `andesite_core` wires the four planes at the documented wiring table. RTL stands at 37 SystemVerilog files (31 fub, 4 plane, 1 top, package). Suites: fub 225, macro 19 (scheduler 10, training 6, DFI smoke 1, axi4 parity 1, init slice 1), core top 4 — 248 tests green; master Verilator lint (53 modules) and the filelist registry PASS. Two implementation findings belong to this history: the macro's registered init request outlived its own accept cycle (every init command double-accepted) — fixed by gating combinationally on the accept cycle; and the A10 address bit was being dropped by a wrapper that narrowed the column to COL_WIDTH. DV-framework note in the 0.2 tradition: the external DFI BFM's behavior dispatch now samples `cs_n`/error/init/update

Version	Date	Change
		wires unconditionally, so the P1 init-slice TB declares the legacy `phy_dfi_cs_n` name and the idle-tied handshake pins. LPDDR4 breadth remains doc-level by design (P1 init is DDR4-only; the CA submodule is the recorded TASK-008 follow-on).
0.1	2026-10-03	First edition, from the 2026-10-03 bootstrap spec. Written before RTL, the way scoria's 0.1 was: the delta analysis settles the markings, the design point, and the DFI 4.0 boundary; the MAS and the kmap book follow as separate books.
0.4	2026-10-04	P3 gate per spec §3: the modified/new tier is landed — FGR refresh (1x/2x/4x, reload-only), the L/S scheduler pair (bank-group-aware admission, LPDDR4 by degeneration), the LPDDR4 MPC ZQ submodule and CA/WDQ training interfaces (opcode images CSR-driven, encodings TBC per JESD209-4 — nothing

Version	Date	Change
		invented), dynamic ODT policy per the MAS 08 fence, DBI on the DFI write-mask and read-data paths per the TASK-005 study (write CRC explicitly inert), and the TASK-006 CA-parity recovery FSM beside the init FSM. Inventory: 30 design files (14 carried-inherited rename-only incl. the dormant pair, 9 carried-modified, 4 new, 3 P1). Suites green across the tier; the HAS/MAS marking-pair gate is GREEN (23=23, markings copied not re-derived). `dfi_cmd_path` and the macro init integration ride the closing pass; `dfi_plane`/top are P4.
0.3	2026-10-04	P2 carried per this chapter's inheritance table: `bank_timer`/`bank_timers`, `page_policy`, the read/write CAMs, `wr_splitter` + `axi_burst_chopper`, `rd_return_ring`, `dfi_cdc`, and `cmd_history_checker` land as `andesite_*` with scoria's suites ported green (the checker grows

Version	Date	Change
		the DDR4 long/short spacing set the MAS names — additive checks, mechanism untouched). `powerdown_ctrl`/`dfi_signal_pack` ride as the dormant pair, uninstantiated, waking condition unchanged. `rd_intake`/`wr_intake` re-sequence to the front of P3: they instantiate `addr_mapper`, a P3 block — dependency order over this book's idealized list, recorded in the P2 gate.
0.2	2026-10-04	P1 RTL landed per the RTL-bootstrap spec: `andesite_pkg`, `andesite_dfi_cmd_formatter`, `andesite_mode_register`, and `andesite_init_sequencer` exist with model-first DV. The init slice walks the DFI 4.0 BFM at DDR4-1600 (vendored jedec/ddr4-1600.csv timings) through JEDEC init with the anchored order proven at the pins and zero DRAM-state violations; gear-down entry exercised as a second configuration. The

Version	Date	Change
		kmap book's command-decode citations now point at the RTL lines, so the drift gate is live; per-command boolean SOP verdicts land with the P3 maps. First-consumer BFM gaps the slice exposed and the DV repo fixed: the v4.0 `cs` rename in the receive path and wired roster, and presence-guarded optional wires. The MAS Table 2.2/2.2.2 port tables were reconciled with the landed interface (v4.0 names, 5-bit `op_i`, 3-bit MR index on `cmd_bank`); deferred ports (`dfi_alert_n`, `ca_o`/`ca_valid_o`) are recorded as follow-ons, not stubs.

2.4 How to read this book

Chapter 1 fixes purpose and vocabulary. Chapter 2 is the shape of the controller, with one figure in which every block is colour-marked against scoria — read that figure and you know the size of this project. Chapter 3 is the heart: the ten areas DDR4 and LPDDR4 actually change, and the dormant and deferred lists. Chapter 4 is the DFI v4.0 boundary and what moved from v3.1. Chapter 5 is the package and parameters. Chapter 6 is how it is verified, and what separates this edition from a 1.0.

The caution this document carries by construction. v0.1 is written before the RTL, and a specification written first can be *unbuildable* — that is the risk it carries. The evidentiary rule is the mitigation. The DFI 4.0 specification is now on disk and was studied on 2026-10-04 as andesite TASK-005; DFI 4.0 clause numbers in this edition are real citations, and the study corrections for

v3.1 → v4.0 chip-select renames, the missing `dfi_wrdata_dbi` pin, and the `dfi_init`/`dfi_ca_capture` mismatches are recorded in HAS ch04 and MAS ch03. Where implementation later corrects the specification, the correction belongs in this revision history, not dropped silently.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 Purpose and Scope

3.1 Purpose

andesite is a unified, parameterized memory controller for DDR4 SDRAM and LPDDR4 SDRAM, presenting an AXI4 slave to the host and a DFI v4.0 master to the PHY. It is the third member of a family: pumice covers DDR2/LPDDR2 and is built, measured and running on hardware; scoria covers DDR3/LPDDR3 with RTL verified in simulation. andesite inherits from scoria the same way scoria inherited from pumice.

This document specifies andesite's architecture. It is written to be *locked* before RTL begins, so the implementation has a fixed target; the follow-on RTL bootstrap plan is authored against this document. Success, per the bootstrap spec, is that a reader can trace every block here block-by-block to scoria's books, that the changed and new blocks are covered in depth by the MAS, and that the DRAM command and mode-register encodings are pinned by the generated kmap book.

3.2 Why a delta specification

andesite is not a fresh design. scoria's architecture — itself pumice's, carried through board bring-up, a read-path rebuild, and a formal campaign that found two real bugs — is the starting point, and scoria's 26-FUB inventory, books, CSR flow and verification apparatus are the immediate reuse pool. Re-deriving any of that would discard its evidence.

So this specification is organised as a delta. Every block is marked **INHERITED**, **MODIFIED** or **NEW** relative to scoria, and the marking carries an obligation:

- **INHERITED** means the scoria block is used unchanged, and its verification evidence transfers. If an inherited block changes during implementation, the marking is wrong and this document must be corrected — not the marking quietly dropped.
- **MODIFIED** means a bounded change with a named cause, always a clause of JESD79-4, JESD209-4 or DFI v4.0.
- **NEW** means no scoria counterpart. The new blocks are named in Chapter 2.3 and specified in the MAS.

3.3 What is in scope

The controller between the AXI4 slave port and the DFI v4.0 master port: the front end, the scheduler, bank and global timing enforcement, refresh, mode registers and initialization, ZQ calibration, training, on-die-termination control, the data paths, and the CSR block. LPDDR4's deltas are carried per-chapter, the same one-book-both-memtypes shape scoria uses.

3.4 What is out of scope

The PHY. DFI is the boundary, as it is for scoria, and for the same reason: IOB serdes, bitslip and delay-line calibration are FPGA- or process-specific and do not belong in a portable controller. This is a deliberate architectural commitment, not an omission.

Board bring-up. Verification is against a DFI v4.0 bus functional model in simulation, and that is the verification boundary. There is deliberately no board named: the 7-series targets this repo's boards carry do not support DDR4, so a board target would be a promise this generation can't keep.

The training searches. DDR4 adds read leveling alongside write leveling, and LPDDR4 adds CA and WDQ training. andesite provides the interfaces — the DFI handshakes, the MR paths, the timing windows, the telemetry — and the search algorithms run in firmware, exactly as scoria's write-leveling search does (decision D2 there, carried here).

LPDDR4 DVFS and deep-sleep states. Named, deferred, and recorded in Chapter 6 with the condition that un-defers them.

The DDR5 half of DFI v4.x. Targeting v4.0 brings signals andesite has no use for — the DDR5-oriented pieces of the 4.x revision family. They are named in Chapter 4 so their absence is a decision on the record rather than an oversight, and they are out of scope the same way scoria's DDR4/v3.1 surface was: named, then deliberately left unimplemented.

The PRD and the RTL. The PRD rewrite and the RTL bootstrap are follow-on work, tracked in the vault lane (`vault/Tasks/projects/components/mem-ctrl-ip/research-ip/andesite-ddr4-lpddr4/`). This book is the first of three: the MAS and the kmap book follow it, each owner-reviewed before the next starts.

4 Document Conventions

4.1 Block marking

Every block carries exactly one marking, and the block diagram colours match:

Table 1.1: Block markings

Marking	Colour in Figure 2.1	Meaning
INHERITED	green	scoria's block, used as-is; its verification evidence transfers
MODIFIED	amber	a bounded change with a cited cause
NEW	red	no scoria counterpart

The marking is a property of the block, not of the chapter it appears in: the Chapter 2 table, the Chapter 3 prose, the Figure 2.1 colours, and the MAS inventory all carry the same marking for the same block, and a drift between those surfaces is a defect in this document.

4.2 What this book does not restate

The family docs at `../../../../docs/` own the shared-core design and the family doctrine — the `mem_ctrl_pkg` design, the config-not-param rule, the maintenance request/grant discipline, the AXI4 host-side shape, and the evidentiary rule itself. This book references them and follows them; it does not restate them. Where this book and a family doc disagree, the family doc wins and this book is the defect.

4.3 Signal and parameter naming

Inherited from the scoria design requirements and not restated here: module naming, `UPPER_CASE` parameters, `r_`/`w_` internal prefixes, and the per-area clock and reset names. Two points bear repeating because they are load-bearing:

- **Clock and reset names are per area.** In the component tree the controller uses `aclk`/`aresetn`. An `.i_rst_n(...)` connection against an `rtl/common` module will not elaborate, because those use `clk`/`rst_n`.
- **No assertions in the RTL.** Properties live in external `formal/` blocks. This is a tool-compatibility rule, not a style preference.

4.4 Timing values

Every enforced timing is a runtime CSR, not a parameter. This is inherited from pumice through scoria and was learned there: a timing compiled in cannot be swept, and a controller whose timings cannot be swept cannot be characterized. andesite keeps it absolutely — geometry alone is build-time (Chapter 2.4).

Where this document states a numeric timing it is cited to a clause, and the unit is given. `nCK` means DRAM clock cycles. Values that vary by speed bin are named but not tabulated here — they belong to the CSR derivation.

4.5 Citations

A claim traceable to a standard is cited as JESD79-4 (DDR4), JESD209-4 (LPDDR4), or DFI v4.0 with a section number. Those PDFs are **not in the repository** and must not be added to it — they are not redistributable, and the DFI 4.0 spec is additionally not on disk at all. Until andesite TASK-005 acquires and studies it, every DFI 4.0 clause citation is suffixed `§TBC(TASK-005)`; the suffix is a promise to confirm, not a reference.

A claim inherited from scoria cites the scoria HAS or its design requirements, both of which are in the repository. A claim about the shared core or the family doctrine cites the family docs.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 Definitions and Acronyms

5.1 Interfaces

AXI4 The host-side slave interface. Inherited from scoria unchanged, including the width-gearing converters that let the host bus width differ from the DRAM's.

DFI DDR PHY Interface. andesite targets **v4.0** — the DDR4-era revision that adds `dfi_act_n`, the gear-down handshake, CA parity and `dfi_alert_n`, and the DBI wires. Clause citations carry `§TBC(TASK-005)` until the spec is on disk.

APB The CSR access interface, through a PeakRDL-generated register block.

5.2 DRAM commands this document refers to

Table 1.2: Commands, and which are new to this family member

Command	In scoria's DDR3/LPDDR3?	Note
`ACT`, `RD`, `WR`, `PRE`, `REF`, `MRS`, `ZQCS`/`ZQCL`, `SRE`/`SRX`	yes	inherited handling; encoding moves to the ACT_n/BG form
`RDA` / `WRA`	yes	read/write with auto-precharge
`PREA`	yes	precharge-all; DDR4 keeps its own encoding
`REF` 1x/2x/4x	1x only	fine-granularity refresh, selected in MR3
`MRS` target set	MR0-MR3 (DDR3), MR0-MR11 (LPDDR3)	DDR4: MR0-MR6; LPDDR4: MR0-MR22 (used subset in Ch 3.2)
`MPC`	**no**	LPDDR4 multipurpose command — ZQ, training, and more over the CA bus
gear-down entry	**no**	DDR4 1/2-rate CA bus mode, entered after init
parity-aware commands	**no**	DDR4 CA parity; a parity error raises `alert_n`

5.3 Terms new to andesite

Bank group DDR4's partition of its 16 banks into 4 bank groups of 4 banks. Bank groups exist to relax same-bank timing constraints: back-to-back commands to different bank groups of the same bank number observe the short constraints, and same-group observe the long ones. LPDDR4 has no bank groups — its 8 banks per channel stand alone.

Table 1.3: The long/short timing pairs bank groups introduce

Parameter	Governs
`tCCD_L` / `tCCD_S`	CAS-to-CAS delay: long, within one bank group; short, across bank groups
`tRRD_L` / `tRRD_S`	ACT-to-ACT delay, same split: long within a bank group, short across

Both pairs are runtime CSRs, like every other timing.

Fine-Granularity Refresh (FGR) DDR4's choice of refresh density, 1x/2x/4x, selected in MR3 (JESD79-4). 1x is scoria's inherited all-bank refresh; 2x and 4x trade more frequent, shorter refresh bursts against the tREFI budget. andesite's `refresh_ctrl` is MODIFIED to carry the selection; scoria's landed TASK-001 modes (elastic refresh, TCR, ZQCS placement) remain the policy base.

Gear-down DDR4's 1/2-rate command/address bus mode. The controller inits at full rate, programs MR3, then switches; DFI v4.0 defines the controller/PHY handshake for the switch `§TBC(TASK-005)`.

CA parity DDR4 command/address parity. The controller counts commands and generates the parity bit; the DRAM checks it and signals errors back on `dfi_alert_n`. How much of the parity machinery lands in hardware versus firmware is an open question (Chapter 6).

DBI (Data Bus Inversion) Per-byte inversion of the data bus to limit switching. DDR4 puts read DBI in MR5 with write DBI alongside; DFI v4.0 carries `dfi_dbi_*` wires `§TBC(TASK-005)`. The DFI datapath block is MODIFIED to pass DBI through.

MPC (Multipurpose Command) LPDDR4's single command opcode that carries what DDR4 spreads across several: ZQ calibration, training steps, and feature mode entry, all addressed over the 6-bit CA bus (JESD209-4). andesite's `zq_ctrl` grows a NEW MPC path for it.

Controller-directed refresh (LPDDR4) LPDDR4's per-bank refresh is directed by the controller — the controller names the bank, unlike DDR4's FGR where the DRAM refreshes what it chooses inside the all-bank command. scoria already handles per-bank refresh for LPDDR3; the LPDDR4 delta is that per-bank becomes the commodity default, not the option.

5.4 Terms used with a specific meaning here

Maintenance traffic Commands no host requested: refresh and ZQ calibration. Because it competes for the command bus, it is the scheduler's concern and not only the sequencer's. The family doctrine binds how it arbitrates: request/grant, and it never preempts demand traffic.

Demand traffic Reads and writes originating from the AXI4 port.

Firmware-driven Performed by software over the CSR interface rather than by a hardware state machine. scoria's write-leveling search is firmware-driven; andesite's read leveling and LPDDR4 CA/WDQ training follow it, per the D2 precedent.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

6 Scope and Goals

6.1 What andesite is

A memory controller presenting an AXI4 slave to the host and a DFI v4.0 master to the PHY, supporting DDR4 SDRAM and LPDDR4 SDRAM from one parameterized design. It is the third member of a family: pumice covers DDR2/LPDDR2 and is built, measured and running on hardware; scoria covers DDR3/LPDDR3 with RTL verified in simulation (221 tests, 9 formal blocks at last measure). andesite inherits from scoria the same way scoria inherited from pumice.

6.2 Goals

1. **Reuse the scoria architecture, and its evidence with it.** The front end, scheduler, timing enforcement and data paths are inherited. Reuse here is not laziness; those blocks carry simulation verification and formal proofs that a rewrite would discard.
2. **Change only what the standards force.** Ten areas change, and the modified set is thirteen blocks, not a rewrite. Chapter 3 is that list, and it is short on purpose.
3. **Keep the DFI boundary clean.** No PHY responsibilities migrate into the controller — including every training *search*, which DDR4 and LPDDR4 make tempting and decision D2's precedent rules out.
4. **Every enforced timing a runtime CSR.** Geometry alone is build-time (Chapter 2.4); everything that varies — including the long/short timing pairs that DDR4's bank-group structure introduces — is a runtime register. Inherited from the family doctrine, and the precondition for characterization.
5. **Verifiable before hardware.** The target is a DFI v4.0 bus functional model in simulation. No board is named, because the 7-series targets this repo's boards carry do not support DDR4; naming one would promise what this generation can't keep.

6.3 Non-goals

Performance parity with a vendor controller at DDR4 speeds. andesite's design point is architectural clarity and measurability, the same posture scoria took. No throughput target is set in this edition.

The DDR5 surface of DFI v4.x. The 4.x revision family carries DDR5 features andesite has no use for. They are named in Chapter 4 so their absence is a decision on the record, then left unimplemented, as scoria left v3.1's DDR4 surface.

Write CRC. DDR4 defines it; this edition excludes it with a named unblock condition (Chapter 3.5). It is a datapath addition, not an architecture change.

LPDDR4 DVFS and deep-sleep states. Named and deferred with the condition recorded in Chapter 6.

Multi-rank beyond scoria's parameterization. `NUM_RANKS` is inherited; the practical limit is the board, not the controller.

6.4 What makes DDR4/LPDDR4 easier than it looks

Three things reduce the work, and stating them keeps the project from being over-scoped:

- **LPDDR4 is not a protocol break from LPDDR3.** It stays a per-bank-refresh, CA-bus family; the deltas are the 6-bit DDR CA bus, MPC, and training — not a new command philosophy. scoria already handles per-bank refresh for LPDDR3; LPDDR4 makes that the commodity default.
- **scoria was written expecting this.** Its package note names the shared `mem_ctrl_pkg` migration as the DDR4/LPDDR4 controller's job (family doc 01), and its refresh policy base (elastic refresh, TCR, ZQCS placement — scoria TASK-001's landed Modes A/B/C) is exactly what FGR wants to sit on.
- **Bank groups are a scheduling change, not a datapath change.** They add a decode dimension and long/short timing pairs; the data paths don't care.

6.5 What is genuinely harder

- **Bank-group scheduling.** tCCD_L/S and tRRD_L/S make the arbiter and the global timers aware of which bank group a command targets — the one structural change DDR4 forces on the scheduler.
- **Dynamic ODT.** RTT_NOM/WR/PARK and the ODT latencies are new machinery with a new block (`odt_ctrl`), and scoria's static handling is insufficient.
- **Read leveling and LPDDR4 CA training.** New interfaces spanning controller, DRAM mode registers and PHY — the write-leveling lesson a generation later.
- **The init sequence.** Reset procedure, MR0-MR6, gear-down entry and parity enable in one ordered sequence; pumice's init-order correction (an EMRS3-

first sequence, benign but wrong) is the precedent for treating the order as JEDEC's, not ours.

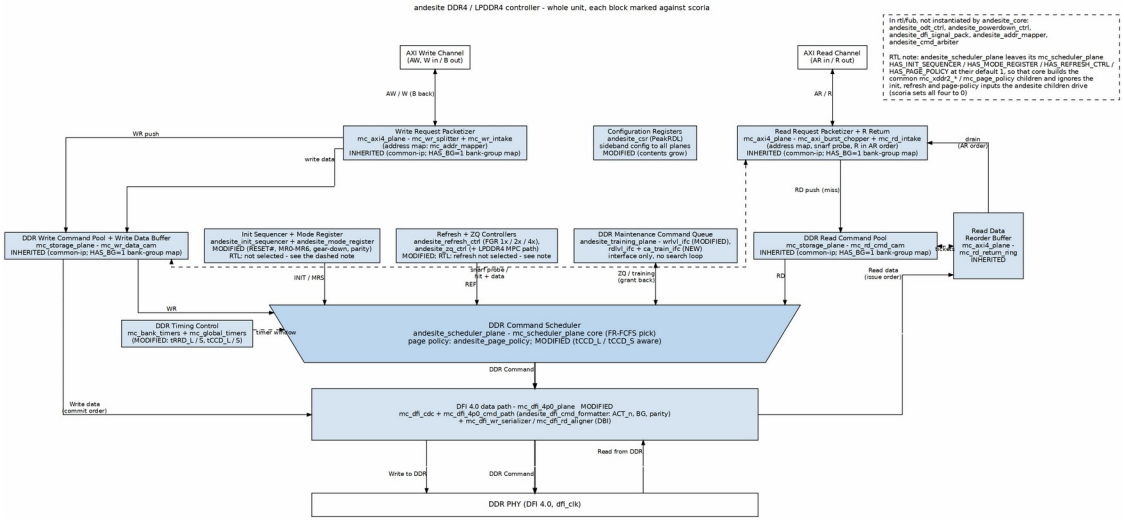
---

7 Block Diagram

7.1 The whole controller, marked against scoria

Read this figure first. The last line of each block's label tells you the size of this project: INHERITED blocks are scoria's (the common-ip planes), used as-is; MODIFIED blocks change in named, bounded ways; NEW blocks are new. The dashed box lists the andesite FUBs that are in `rtl/fub` but not instantiated by `andesite_core` (see the RTL check below).

7.1.1 Figure 2.1: andesite block diagram, with inheritance marking



andesite DDR4/LPDDR4: each block marked INHERITED, MODIFIED or NEW against scoria

Source: 01_block_diagram.dot

7.2 Reading the figure

The front end is entirely INHERITED. The AXI4 interface — read and write intakes, the write-data CAM's front side, the burst chopper, the FSM-free split/aggregate — is untouched by the move to DDR4. Nothing about DDR4 or LPDDR4 reaches the host side.

The scheduler is INHERITED in structure, MODIFIED at the edges. The FR-FCFS arbiter mechanism, the per-(rank,bank) bank timers and the page policy are inherited. What turns MODIFIED is bank-group awareness: the address mapper grows the BG decode, the arbiter and the global timers learn the long/short pairs, and the scheduler keeps admitting maintenance traffic the way scoria's does — request/grant, never preempt (family doctrine 2).

Important: one inherited detail must be inherited *including its fix*. scoria's `global_timers` inherited pumice's next-state-function fix (pumice ISSUE-018: readiness flags published one cycle

late, which permitted tCCD and tRTW violations). andesite inherits the fixed form the same way; a fresh implementation of the same block from the same description would reintroduce the bug.

The MODIFIED maintenance block is refresh, for FGR. `refresh_ctrl` is MODIFIED because DDR4's fine-granularity refresh (1x/2x/4x, selected in MR3) adds a density dimension on top of the mechanism scoria landed with TASK-001 (elastic refresh, TCR, ZQCS placement — the policy base andesite keeps, Ch 3.4). `zq_ctrl` stays INHERITED for DDR4 — DDR4 keeps `ZQCS`/`ZQCL` exactly as scoria issues them — with a NEW submodule for LPDDR4, whose calibration rides the MPC command.

The NEW blocks are three, and all three are bounded. `odt_ctrl` is new because dynamic ODT has no scoria counterpart. `rdlvl_ifc` and `ca_train_ifc` are new because read leveling and LPDDR4 CA/WDQ training have none. All three are interfaces with telemetry; none contains a search loop.

`dfi_signal_pack` is carried dormant. The pack stage stays a registered pipeline; its DFI 4.0 frequency-ratio variants go unexercised at this design point. The waking conditions for it and `powerdown_ctrl` are named in Chapter 3.1.

RTL check (2026-10-13). The figure is drawn from `andesite_core`, and two things in it differ from the text above. First, `andesite_odt_ctrl`, `andesite_powerdown_ctrl`, `andesite_dfi_signal_pack`, `andesite_addr_mapper` and `andesite_cmd_arbiter` exist in `rtl/fub` but nothing in `andesite_core` instantiates them: the pick is the common `mc_cmd_arbiter`, the address map is the common `mc_addr_mapper` (with `HAS_BG=1`), and the DFI pins come from `mc_dfi_4p0_plane`. The figure lists them in its dashed box. Second, `andesite_scheduler_plane` does not override the `HAS_INIT_SEQUENCER`, `HAS_MODE_REGISTER`, `HAS_REFRESH_CTRL` and `HAS_PAGE_POLICY` parameters of its `mc_scheduler_plane` core, which default to 1, so the core builds the common `mc_xddr2_*` and `mc_page_policy` children and ignores the init, refresh and page-policy inputs the andesite children drive. `scoria_scheduler_plane` sets all four to 0.

7.3 What the figure deliberately omits

The PHY, because DFI is the boundary. Any training-search state machine, because the searches live in firmware per the D2 precedent. The DDR5-oriented signals of DFI v4.x, because andesite does not implement them. Each absence is named somewhere in this book so it reads as a decision rather than an oversight.

8 Module Hierarchy

8.1 Planned hierarchy

andesite mirrors scoria's three-tier shape — FUBs under macros under a top — with the `andesite_` prefix replacing `scoria_`. The tables below are the planned module list with its marking; they are a specification, and the RTL is expected to match them or this document to be corrected. The accounting counts scoria's FUB files as they exist on disk — 26 of them — and the markings sum to that number.

8.1.1 Top and macro tier

Table 2.1: Top and macro tier

andesite module	scoria counterpart	Marking
`andesite_top`	`scoria_top`	INHERITED (structure)
`andesite_top_geared`	`scoria_top_geared`	INHERITED — the host/DRAM width-gearing wrapper
`andesite_core`	`scoria_core`	INHERITED
`andesite_axi4_plane`	`scoria_axi4_ifc`	INHERITED
`andesite_scheduler_plane`	`scoria_mem_cmd_scheduler`	MODIFIED — bank-group-aware L/S admission, on top of scoria's ZQ maintenance admission
`andesite_training_plane`	—	NEW — holds the three training FUBs, owns the DFI training pins + one-active mux, and issues MRS/MPC on the scheduler's maintenance training channel
`andesite_dfi_plane`	`scoria_dfi_plane`	IMPLEMENTED — DFI 4.0 control surface, DBI wires (TASK-016 t6)

The CSR block rides at the macro tier as scoria's does: an `andesite_csr.rdl` generated through the same PeakRDL flow. The **flow is INHERITED**; the **contents grow** (FGR select, ODT policy, DBI enable, gear-down, parity) — Chapter 5 owns the parameter and CSR inventory.

8.1.2 FUB tier, inherited unchanged

`bank_timer` and `bank_timers`, `page_policy`, `rd_cmd_cam`, `wr_data_cam`, `rd_intake`, `wr_intake`, `wr_splitter`, `rd_return_ring`, `axi_burst_chopper`, `dfi_cdc`, and `cmd_history_checker`.

That is 12 of the 26 scoria FUB files carried over with no functional change. Two more ride along in a grey area, each with its condition named in Chapter 3.1: **`powerdown_ctrl` DORMANT** — power-down is CKE plus SRE/SRX, unchanged in mechanism — and **`dfi_signal_pack` DORMANT**, the registered pack stage. A thirteenth, **`zq_ctrl` INHERITED** for DDR4 exactly as scoria built it, with a NEW submodule for LPDDR4's MPC calibration path (Chapter 3.2).

`cmd_history_checker` is worth naming explicitly: it is the verification-side block that independently re-derives JEDEC spacing from the issued command stream. Its mechanism is inherited; its spacing-parameter set grows DDR4's long/short pairs, and that growth is a parameter change, not a mechanism change — the same disposition scoria recorded for DDR3's parameters.

8.1.3 FUB tier, modified

Table 2.2: Modified FUBs — every row in this table is MODIFIED, and the marking column says so rather than leaving it to the section heading

Module	Marking	Why it changes	Detail
`addr_mapper`	MODIFIED	bank-group decode: BG0/BG1 sit between chip select, bank and row	Ch 3.1
`cmd_arbiter`	MODIFIED	L/S-aware issue — same-bank-group pairs take tCCD_L/tRRD_L, cross-group take tCCD_S/tRRD_S	Ch 3.1
`global_timers`	MODIFIED	the long/short pairs join the enforced timing set as first-class	Ch 3.1

Module	Marking	Why it changes	Detail
		counters	
dfi_cmd_formatter	MODIFIED	ACT_n fifth command pin, BG0/BG1, CA parity; NEW submodule: the LPDDR4 6-bit CA path	Ch 3.2, 3.6
init_sequencer	MODIFIED	DDR4 reset procedure, MR0-MR6 order, gear-down entry, parity enable	Ch 3.2
mode_register	MODIFIED	MR0-MR6 field maps per memtype	Ch 3.2
refresh_ctrl	MODIFIED	FGR 1x/2x/4x via MR3 on the inherited elastic/TCR/placement base (scoria's landed TASK-001 Modes A/B/C)	Ch 3.4
dfi_cmd_path	MODIFIED	ACT_n, BG and parity wires ride the command path to the PHY	Ch 3.2, 4.1
dfi_rd_aligner	MODIFIED	DBI read path passes the inversion through	Ch 3.5
dfi_wr_serializer	MODIFIED	DBI write path passes the inversion through	Ch 3.5

Module	Marking	Why it changes	Detail
wrlvl_ifc	MODIFIED	the training-interface family grows a read-leveling sibling; still no search loop	Ch 3.3

8.1.4 FUB tier, new

Table 2.3: New FUBs — every row is NEW

Module	Marking	Purpose
odt_ctrl	NEW	dynamic ODT: RTT_NOM/WR/PARK selection, ODT latency enforcement, ODT telemetry. **No scoria counterpart** — scoria holds ODT static, which DDR4 makes insufficient (Ch 3.5)
rdlvl_ifc	NEW	MPR-based read-leveling interface: DFI handshake, MR3 path, timing windows, telemetry. **No search loop** (Ch 3.3)
ca_train_ifc	NEW	LPDDR4 CA/WDQ training interface over the CA bus: DFI handshake, MPC paths, telemetry. **No search loop** (Ch 3.3)

8.2 Naming, and the package

Per the family doc 01, andesite carries its own andesite_pkg from day one, and the three near-identical family packages (pumice's, scoria's, andesite's) are deliberate and time-boxed: the

shared `mem_ctrl_pkg` migration executes once, when andesite RTL bring-up starts, and both shipping controllers move together. Chapter 5 owns `andesite_pkg`'s initial contents; the family doc owns the migration plan and its conditions. Nothing about the package duplication is an oversight, and nobody should "fix" it by widening a shipping CSR.

---

9 The Target Design Point

9.1 Named as of v0.1

There is deliberately no board. The 7-series targets this repo's boards carry do not support DDR4, and a named board would be a promise this generation can't keep. So the design point is stated as a **simulation target**: the geometry, the data rates and the DFI boundary the parameters are chosen for, verified against a DFI v4.0 bus functional model (Chapter 6). What follows is what a future board target must match, not a claim that a bitstream exists.

Table 2.4: The target design point

Item	Value	Source
DRAM, DDR4 side	DDR4-1600 x8, 4 bank groups × 4 banks = 16 banks, MT40A1G8-class	bootstrap spec §2 design point
DRAM, LPDDR4 side	LPDDR4-1600 x16, 8 banks per channel — ungrouped; the package offers two x16 channels	bootstrap spec §2; JESD209-4
Data rate	1600 MT/s both memtypes — 800 MHz CK, tCK 1.25 ns	arithmetic from the rate
PHY boundary	DFI v4.0, frequency ratio 1:4 carried from scoria	Ch 4
Verification	DFI v4.0 BFM in simulation (BFM acquisition/study = andesite TASK-005)	Ch 6
Theoretical peak, DDR4 side	**1600 MB/s** per x8 device — 1600 MT/s x 8 bit / 8	arithmetic; the denominator for any bandwidth claim
Theoretical peak, LPDDR4 side	**3200 MB/s** per x16 channel — 1600 MT/s x 16 bit / 8	arithmetic; same rule

The denominator rule applies from this page on. Any bandwidth figure quoted anywhere in this book or its successors carries the theoretical maximum beside it. pumice learned this twice —

a byte counter that was per-phase, and a percentage with no peak beside it — and the family doctrine keeps the lesson.

9.2 What this fixes

Build-time parameters the architecture left open now have values; everything else stays a runtime CSR:

Table 2.5: Build-time parameters fixed by the design point

Parameter	Value	Why it is forced
DFI frequency ratio	1:4	carried from scoria — the datapath's phase structure transfers at the gear it was verified at
DDR4 geometry	per Table 2.4	the DDR4 design point; build-time because it sizes decode, not because it varies
LPDDR4 geometry	per Table 2.4 — ungrouped	the LPDDR4 design point; same rule
DQ width	x8 (DDR4) / x16 (LPDDR4)	the design point's devices
`NUM_RANKS`	1	single-rank design point, as scoria's was

Note what is *not* in the table: every timing. tCCD_L/S and tRRD_L/S are runtime CSRs exactly like their plain ancestors, and the JEDEC speed-bin values that initialise them belong to the CSR derivation (Chapter 5), cited to JESD79-4 / JESD209-4 — the standards live in cold storage, not the repo, per the house rule.

Warning — the cycle counts in that CSR derivation are only correct at the MC clock frequency they were derived for. Every timing CSR is a count of controller-clock cycles, not nanoseconds. Change the MC clock frequency and the whole timing set — JEDEC timings, init and ZQ waits, training windows, ODT latencies — must be re-derived at the new frequency and reprogrammed before traffic resumes. A stale value violates a real DRAM timing and presents as data corruption, not as an error. This is a binding property of the register block being specified, not an implementation choice; no hardware frequency-change protocol is specified, so reprogramming is software's job.

The 1:4 ratio is the same happy accident scoria had. pumice and scoria both run four DFI phases, so the inherited DFI datapath — the async-FIFO CDC, the per-phase pipelines — transfers

at the gear it was built and verified at. DFI 4.0’s frequency-ratio options beyond 1:4 are named in Chapter 4 and left for a board PHY to justify.

9.3 What is still open

Nothing in the design point itself. Three adjacent choices are recorded as open questions in Chapter 6 rather than decided here: CA-parity scope (hardware counter vs firmware assist), the gear-down coverage scope for the BFM, and LPDDR4 channel count beyond the one-channel design point.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

10 What Is Inherited, and the Ten Areas That Change

10.1 The inheritance, stated once

Twelve of scoria's twenty-six FUB files carry over with no functional change, plus the top, geared wrapper, core and AXI4 macro — Chapter 2.3 lists them and the count is reconciled there. This chapter covers only what changes: ten areas, absorbed by a modified set of thirteen blocks and three new ones. The list is short by design, and it is an argument, not a convenience — the inherited blocks carry scoria's verification evidence (221 tests, 9 formal blocks at last measure), and re-deriving them would discard it.

Important — inherit `global_timers` in its FIXED form. scoria's timers carry pumice's next-state-function fix (pumice ISSUE-018: readiness flags one cycle late, which permitted tCCD and tRTW violations). andesite inherits the fixed form; a fresh implementation from a behavioural description would reintroduce the defect, because the description does not mention the flop. Copy the module.

10.2 The changes, in one table

The ten areas the bootstrap spec settled, with the blocks that absorb them:

Table 3.0: The ten delta areas, verbatim from the bootstrap spec

#	Delta	Blocks	Marking
1	Bank groups: tCCD_L/S, tRRD_L/S	`addr_mapper`, scheduler/`cmd_arbiter`, `global_timers`	MODIFIED
2	Command encoding: ACT_n 5th pin, BG0/1; LPDDR4 6-bit CA bus	`dfi_cmd_formatter`	MODIFIED / NEW (LPDDR4 path)
3	Init: reset procedure, MR0–MR6, gear-down + parity enable	`init_sequencer`, `mode_register`	MODIFIED
4	Refresh: FGR 1x/2x/4x (MR3); LPDDR4	`refresh_ctrl`	MODIFIED (inherits scoria's landed TASK-001

#	Delta	Blocks	Marking
	controller-directed per-bank		modes A/B/C as the policy base)
5	ZQ: DDR4 keeps ZQCS/ZQCL; LPDDR4 calibration via MPC	`zq_ctrl`	INHERITED / NEW (LPDDR4 path)
6	Training: read leveling (MPR), LPDDR4 CA/WDQ training	new training blocks + `wrlvl_ifc`	NEW / MODIFIED (search in firmware, per D2 precedent)
7	Datapath: DBI via DFI 4.0 `dfi_dbi_*`; write CRC excluded v0.1 with named unblock condition	dfi datapath blocks	MODIFIED
8	CA parity + `alert_n`; gear-down handshake	formatter/ sequencer + new parity/alert handling	MODIFIED / NEW
9	Dynamic ODT: RTT_NOM/WR/PARK + ODT latencies (scoria holds ODT static — insufficient)	new `odt_ctrl`	NEW
10	LPDDR4 per-chapter deltas: no bank groups, 8 banks/channel, 2-channel x16, MPC bus, DSM power states (deferred, named condition)	as above	as above

The command encodings, decode maps, MR maps, ODT policy, and FGR select named by the ten areas above are pinned table-by-table in the generated kmap book (`../../kmaps/generated/`). The DDR4 command table's expected values feed the Chapter 6 verification item 6 checker.

10.3 The ten areas, one paragraph each

1 — Bank groups. DDR4 partitions its banks into groups, and command spacing becomes a function of which group a command targets: tCCD_S/tRRD_S across groups, tCCD_L/tRRD_L within one. The address mapper grows the BG0/BG1 decode, the arbiter learns same-group versus cross-group issue, and the global timers take the long/short pairs on as first-class counters. All four timings are runtime CSRs. LPDDR4 has no bank groups — its pair collapses to L = S, which the arbiter must degenerate to gracefully, not special-case. **Blocks:** `addr_mapper`, `cmd_arbiter`, `global_timers`, `scheduler_plane` — all MODIFIED (Ch 3.1).

2 — Command encoding. DDR4 makes ACT a five-pin command: `ACT_n` plus RAS/CAS/WE, and carries BG0/BG1 on the address pins during activate. The formatter's truth table changes shape, and the command path widens to carry the new pins. LPDDR4 is the bigger change inside the same block: a 6-bit double-data-rate CA bus with commands taking two cycles, implemented as a NEW submodule so the DDR4 path stays reviewable on its own. **Blocks:** `dfi_cmd_formatter` MODIFIED with a NEW LPDDR4 CA submodule (Ch 3.2, 3.6).

3 — Init. DDR4 brings a RESET# pin, a seven-register mode-register set, gear-down entry and parity enable into one ordered sequence. scoria's lesson applies twice over: the MR order is JEDEC's, not ours. **Blocks:** `init_sequencer`, `mode_register` — MODIFIED (Ch 3.2).

4 — Refresh. DDR4's fine-granularity refresh (1x/2x/4x, selected in MR3) adds a density dimension; LPDDR4 makes per-bank refresh the commodity default and — unlike LPDDR2/3 — lets the controller name the bank. scoria's landed TASK-001 modes (elastic refresh, TCR, ZQCS placement) stay the policy base. **Block:** `refresh_ctrl` MODIFIED (Ch 3.4).

5 — ZQ. DDR4 keeps `ZQCS`/`ZQCL` exactly as scoria issues them — the block is inherited unchanged for DDR4. LPDDR4 moves calibration onto the MPC command, a NEW submodule beside the inherited core. **Block:** `zq_ctrl` INHERITED / NEW LPDDR4 path (Ch 3.2).

6 — Training. Write leveling's interface is inherited; read leveling (MPR-based) and LPDDR4 CA/WDQ training are new interfaces in the same family — handshake, MR/MPC paths, timing windows, telemetry, and **no search loop anywhere** (D2 precedent). **Blocks:** `wrlvl_ifc` MODIFIED; `rdlvl_ifc`, `ca_train_ifc` NEW (Ch 3.3).

7 — Datapath. DFI 4.0's `dfi_dbi_*` wires pass Data Bus Inversion through the read and write paths. Write CRC is excluded from this edition with a named unblock condition below. **Blocks:** `dfi_rd_aligner`, `dfi_wr_serializer`, `dfi_plane` — MODIFIED (Ch 3.5, 4.1).

8 — CA parity and gear-down. DDR4 CA parity adds a counting/checking obligation and the `alert_n` return path; gear-down halves the CA rate after init with a controller/PHY handshake. Parity machinery splits between the formatter (counting) and the sequencer (enable at init);

`alert_n` handling is a NEW submodule at the DFI boundary. Scope split between hardware and firmware is an open question (Ch 6). **Blocks:** `dfi_cmd_formatter`, `init_sequencer` MODIFIED; NEW `alert_n` handling (Ch 3.2, 4.1).

9 — Dynamic ODT. DDR4's termination is dynamic: RTT_NOM, RTT_WR during write bursts, RTT_PARK when idle, with ODT-pin latencies around each transition. scoria holds ODT static — that is insufficient the moment writes and reads interleave, and the policy decisions (which ranks terminate when) need a home. **`odt_ctrl` is NEW** (Ch 3.5).

10 — LPDDR4 per-chapter deltas. LPDDR4's deltas ride the chapters above rather than forming an eleventh area: no bank groups (area 1), the CA bus (area 2), MPC-carried ZQ (area 5), CA/WDQ training (area 6), controller- directed refresh (area 4), MR-programmed termination (area 9), and DSM power states deferred with the condition below. Chapter 3.6 collects them as a reading aid.

10.4 The full reuse table

Every scoria module, its andesite marking, and its cause. This table and Chapter 2.3 carry the same markings; where they disagree, one of them is wrong and both get fixed.

Table 3.1: The complete reuse table — every scoria module with its andesite marking

scoria module	Marking	Cause	Where
`scoria_top`	INHERITED	structure unchanged	Ch 2.3
`scoria_top_geared`	INHERITED	the host/DRAM width-gearing wrapper, unchanged	Ch 2.3
`scoria_core`	INHERITED	core assembly unchanged	Ch 2.3
`scoria_axi4_ifc`	INHERITED	the AXI4 host side is untouched by this generation	Ch 2.3, 4.2
`scoria_mem_cmd_scheduler`	MODIFIED	L/S admission alongside scoria's ZQ maintenance admission	3.1
`scoria_dfi_plane`	MODIFIED	DFI 4.0 control surface and DBI	4.1

scoria module	Marking	Cause	Where
		wires	
scoria_addr_mapper	MODIFIED	bank-group decode	3.1
scoria_cmd_arbiter	MODIFIED	L/S-aware issue	3.1
scoria_global_timers	MODIFIED	the long/short pairs	3.1
scoria_dfi_cmd_formatter	MODIFIED	ACT_n, BG0/1, CA parity; NEW submodule: LPDDR4 6-bit CA path	3.2, 3.6
scoria_init_sequencer	MODIFIED	reset procedure, MR0-MR6 order, gear-down entry, parity enable	3.2
scoria_mode_register	MODIFIED	MR0-MR6 field maps per memtype	3.2
scoria_zq_ctrl	INHERITED	DDR4's ZQCS/ZQCL exactly as built; NEW submodule: LPDDR4 MPC calibration	3.2
scoria_refresh_ctrl	MODIFIED	FGR 1x/2x/4x on the inherited elastic/TCR/placement base	3.4
scoria_dfi_cmd_path	MODIFIED	ACT_n, BG and parity wires ride the command path	4.1
scoria_dfi_rd_aligner	MODIFIED	DBI read path	3.5

scoria module	Marking	Cause	Where
`scoria_dfi_wr_serializer`	MODIFIED	DBI write path	3.5
`scoria_wrlvl_ifc`	MODIFIED	training-interface family grows; contract and no-search rule unchanged	3.3
`scoria_powerdown_ctrl`	DORMANT	carried dormant, condition below	below
`scoria_dfi_signal_pack`	DORMANT	carried dormant, condition below	below
(new) `odt_ctrl`	NEW	dynamic ODT	3.5
(new) `rdlvl_ifc`	NEW	MPR read-leveling interface	3.3
(new) `ca_train_ifc`	NEW	LPDDR4 CA/WDQ training interface	3.3

Twelve further FUBs are inherited unchanged and live in Chapter 2.3's prose: `bank_timer`/`bank_timers`, `page_policy`, `rd_cmd_cam`, `wr_data_cam`, `rd_intake`, `wr_intake`, `wr_splitter`, `rd_return_ring`, `axi_burst_chopper`, `dfi_cdc`, and `cmd_history_checker` (whose spacing-parameter set grows DDR4's pairs — a parameter change, not a mechanism change). The CSR block's generation flow is INHERITED; its contents grow and Chapter 5 owns that.

10.5 Dormant: `powerdown_ctrl` and `dfi_signal_pack`

Both are carried dormant, the same disposition scoria reached for the same pair, with the condition named rather than implied. **Waking condition:** a board target exists for andesite, or the LPDDR4 DSM work below un-defers — whichever comes first. Until then both files are kept in the tree as the starting point for that work, worth more dormant than deleted.

The reasoning splits by memtype. For DDR4, power-down is CKE plus SRE/SRX, and the sim-only design point exercises neither meaningfully — the DRAM simply stays in normal operation, which is parity with the references this design point is verified against. For LPDDR4 the pair is load-bearing: deep-sleep states need the DRAM clock stopped (`dfi_dram_clk_disable`), and that is `dfi_signal_pack`'s one function the phase-multiplied buses don't cover. LPDDR4 *exists* for power; deferring its sleep states is a scope decision, not a judgment that they don't matter.

10.6 Deferred, with named conditions

Three items are deliberately out of this edition, each with the condition that un-defers it recorded so the parking is a decision rather than neglect:

Table 3.2: The deferred list

Deferred item	Condition that un-defers it
Write CRC (DDR4, datapath)	A characterization or board campaign that needs end-to-end write protection; the addition is bounded to the write datapath and its CSRs
LPDDR4 DVFS / deep-sleep states	A low-power consumer and a target that can measure power; wakes the dormant pair above
Self-refresh scheduling as a policy	Same waking condition as the dormant pair, plus a workload whose idle behaviour makes the policy a question worth answering

A note on the cleverer refresh schemes. Out-of-order per-bank refresh, write-refresh parallelisation, refresh pausing and SARP/DSARP are surveyed in andesite TASK-001 (it predates this book and its roadmap is the `ADVANCED_MODES_ROADMAP`); this edition implements commodity refresh only and leaves the survey's research items where they are.

11 Init and ZQ: RESET#, MR0-MR6, Gear-Down

11.1 The sequence is JEDEC's, verbatim

JESD79-4 specifies DDR4 power-up initialization as a numbered sequence, and JESD209-4 specifies LPDDR4's own. Both are reproduced here as *requirements on the sequencer*, in each spec's order, because the order is not ours to choose. The timings are named with their JEDEC symbols; their values are cited to the standards' initialization sections and land in the CSR derivation (Chapter 5). What binds the sequencer is the order and the dependencies, and those are stated fully.

DDR4:

Table 3.3: DDR4 initialization requirements, ordered per JESD79-4

Step	Requirement
1	Power ramp, with the spec's voltage-ordering constraints. Board-level, not controller-level
2	Assert `RESET#` through its power-up window (`tINIT1`), then de-assert; keep CKE inactive
3	Wait the RESET#-deassertion-to-CKE interval (`tINIT3`), clocks stable per the spec's requirement
4	CKE active starts the internal init; wait `tINIT4` before the first MRS
5	MRS to **MR3**
6	MRS to **MR6**
7	MRS to **MR5**
8	MRS to **MR4**
9	MRS to **MR2**
10	MRS to **MR1** (DLL enable, and ODT/RTT_NOM programming per the board)
11	MRS to **MR0** (DLL reset)
12	**ZQCL** to start ZQ calibration
13	Wait `tDLLK` and `tZQinit`; ready for

Step	Requirement
	normal operation
14	*If gear-down is used:* program gear-down in MR3, run the gear-down entry sequence, wait its sync time — then normal operation at half CA rate

LPDDR4:

Table 3.4: LPDDR4 initialization requirements, ordered per JESD209-4

Step	Requirement
1	Power ramp with the spec's sequencing; MRW to the reset-related mode states per JESD209-4
2	Reset (performed over the CA bus or the reset pin per the part), then the spec's initialization wait
3	Mode-register writes in JESD209-4's order — including the ODT, drive-strength and DQ ODT programming LPDDR4 carries in its MR set
4	ZQ calibration via **MPC** (the LPDDR4 path — see the ZQ section below)
5	Ready for normal operation; CA training before reliable operation if the board requires it (Ch 3.3)

Two recordings follow the tables, per the evidentiary rule. **First:** the numeric init constants (the `tINIT*` family, `tDLLK`, `tZQinit`, `tMRD`, `tMOD`) are cited to JESD79-4 §3 / JESD209-4's initialization sections and are recorded as open question Q1 in Chapter 6 until read from cold storage at CSR-derivation time — this book does not state them from memory. **Second:** the MRS order above is MR3-MR6-MR5-MR4-MR2-MR1-MR0, and it is not sorted — MR0 carries the DLL reset and goes last, so the order encodes a dependency the way scoria's MR2-MR3-MR1-MR0 did. pumice's EMRS3-first correction is the family precedent: the sequence is a citation, not a design choice.

11.2 What the sequencer owns

RESET# is a pin, not a command — scoria already learned this shape, and andesite inherits the pattern: the sequencer drives a reset output, holds it through its timed window, and the output reaches the top level and the PHY.

Gear-down entry. DDR4 runs its CA bus at full rate for initialization, then optionally halves it. The entry is a sequenced mode switch: program gear-down in MR3, issue the entry sequence, wait the sync interval, then operate at half rate. The DFI side has a matching controller/PHY handshake so both sides of the boundary switch together `§TBC(TASK-005)`. Whether the design point uses gear-down is a CSR-selectable choice; the FSM must support both rates because initialization always happens at full rate.

Parity enable. CA parity is enabled through the MR sequence (MR5 carries parity's programming; the exact fields are the MAS's mode-register chapter). The sequencer's obligation is ordering: parity may only be enabled once the bus is stable, and the formatter's parity counter starts counting from that point — a dependency between the sequencer and the formatter that the FSM states must make explicit.

The mode-register set becomes seven. `mode_register` (MODIFIED) carries MR0-MR6 for DDR4 — MR0 (burst, CAS latency, DLL reset, write recovery), MR1 (DLL enable, additive latency, RTT_NOM, write leveling), MR2 (CAS write latency, RTT_WR, write CRC mode bits — inert this edition, LP ASR), MR3 (**MPR access, FGR select, gear-down**), MR4 (temperature status, preamble, CAL), MR5 (read and write DBI, RTT_PARK, data-mask enable, CA parity latency/mode), MR6 (VrefDQ training) — and the LPDDR4 MR set beside it. Field-level maps are the MAS's business; this chapter binds the register count, the programming order, and which MRs couple to other blocks (MR3 to refresh and training, MR5 to ODT, DBI datapath and parity machinery, MR2 RTT_WR to `odt_ctrl`).

11.3 ZQ calibration

DDR4's ZQ story is inherited. `ZQCL` closes initialization, `tZQinit` gates readiness, `ZQCS` runs periodically as maintenance traffic with its interval a runtime CSR, and the module requests the bus and waits — it never preempts (family doctrine 2). scoria's `zq_ctrl` does exactly this and andesite uses it for DDR4 unchanged, including the CSR-selectable placement policy scoria landed as TASK-001 Mode C.

LPDDR4's is the new path. JESD209-4 carries calibration on the **MPC** command — ZQCal start and latch as MPC opcodes on the CA bus — so `zq_ctrl` grows a NEW submodule: an MPC issuer that sequences calibration through the formatter's LPDDR4 CA path and reports the same way the inherited core does. The maintenance shape is unchanged: interval CSR, request and wait, telemetry count. What changes is only the bus the calibration rides.

The scheduler consequence is one input, as it was in scoria. Refresh demand, ZQ demand (DDR4 or LPDDR4), and the new odt_ctrl's turnarounds all arrive as maintenance-class requests; `scheduler_plane` arbitrates them with the same request/grant machinery it inherited.

12 Training: Write Leveling Inherited, Read Leveling New

12.1 The family rule: interfaces in hardware, searches in firmware

This family has made the training decision once already, and andesite makes the same decision for the same reasons. scoria's D2 settled it for write leveling: the controller provides the *interface* — the DFI handshake, the MR path, the timing windows, the telemetry — and the *search* runs in firmware. A delay-line walk's step size, tap count, monotonicity and temperature behaviour are PHY properties, not controller properties, and a calibration bug should be a script edit, not a respin.

Corroborated by LiteDRAM, once again. The same generated-core check that backed scoria's D2 backs this extension: LiteDRAM's training support is enable-plus-strobe registers with no leveling state machine, and its DDR4 generation adds read-leveling and CA-training knobs in exactly that shape. An independent, deployed controller reaching the same conclusion is worth more than the argument on its own — and it tells us the interface shape that works in practice.

So andesite's training blocks obey three rules, inherited:

1. **No search loop in hardware.** Anywhere. The blocks handshake, sequence, window, and report; firmware walks.
2. **The four-state telemetry rule.** A host must distinguish never attempted, attempted and converged, attempted and timed out, and attempted with no result across the whole window — a detector that has never fired is not evidence of anything (pumice's data-integrity CRC lesson, quoted in full in scoria's book and not repeated here).
3. **Timeout is the controller's to define.** Where a JEDEC window's maximum is controller-dependent, the CSR defines it with a distinct status bit — a host that waits forever for a result it will never get is indistinguishable from a broken link.

12.2 Write leveling: `wrlvl_ifc`, MODIFIED

The interface is scoria's, carried with its contract intact: DFI leveling handshake, MR1 path in and out of the DRAM's leveling mode, the `tWL*` timing windows as runtime CSRs, prime-DQ result capture, and the four-state telemetry. The marking is MODIFIED for one bounded reason: the training-interface family grows a read-leveling sibling, and the shared machinery (handshake discipline, window enforcement, telemetry framing) is factored so both interfaces ride it. What the module owes the host is unchanged.

12.3 Read leveling: `rdlvl_ifc`, NEW

DDR4's fly-by topology skews DQS against CK on the read side too, and the DRAM can help: MR3's **MPR** (multi-purpose register) makes the DRAM drive a known, training- only pattern onto DQ, so the controller can observe its read capture timing without any memory array in the loop.

Table 3.5: `rdlvl_ifc` responsibilities

Responsibility	Detail
DFI read-leveling handshake	request/acknowledge per chip select with the DFI v4.0 read-training signals §TBC(TASK-005)
MPR path	MRW into and out of MPR access mode via MR3, sequenced against the handshake
Timing windows	the MPR readout and pre/post-pattern intervals as runtime CSRs, named per JESD79-4
Result capture	the training pattern's observed value per chip select, presented to the host
Telemetry	the four-state rule above: attempts, results, window, completion

The search is firmware's: enable MPR, capture, walk the read delay, cache the result. The block never decides anything.

12.4 LPDDR4 CA/WDQ training: `ca_train_ifc`, NEW

LPDDR4's bus needs training DDR4 doesn't: the **CA bus itself** must be trained (LPDDR4 samples CA against CK, and both edges matter), and the write-DQ path needs its own calibration step. JESD209-4 carries both through the MPC command's training opcodes — which is why this block sits beside `zq_ctrl`'s MPC submodule in the dependency graph and reuses the formatter's LPDDR4 CA path.

Table 3.6: `ca_train_ifc` responsibilities

Responsibility	Detail
MPC training-opcode issuer	CA training entry/exit and WDQ calibration steps over the 6-bit CA bus
DFI training handshake	LPDDR4's training-mode signalling to

Responsibility	Detail
	the PHY, per DFI v4.0 §TBC(TASK-005)
Timing windows	the JESD209-4 training intervals as runtime CSRs
Result capture	the sampled CA/DQ observation per step, presented to the host
Telemetry	the four-state rule; per-channel because LPDDR4's channels train independently

A scope note. Whether LPDDR4 CA training is *mandatory before operation* is board-dependent; the init chapter (3.2) records it as a conditional step. The interface exists either way — a board that needs it must not require new hardware to use it.

12.5 What training costs the surrounding blocks

The training interfaces touch a handful of other blocks, and the touches are bounded. The new `andesite_training_plane` macro holds the three training FUBs, owns the DFI training pins and the one-active mux, and issues MRS/MPC commands on a dedicated maintenance training channel (`trn_cmd_*`) into the scheduler. The `init_sequencer` gains training-entry states after ZQ (firmware-triggered, not autonomous — init hands the bus to training explicitly). The `mode_register` carries the MR3 MPR fields and the LPDDR4 training-related MRs. The `dfi_cmd_formatter`'s LPDDR4 CA submodule issues the MPC training opcodes. No datapath block changes for training; the read and write paths already carry whatever the PHY loops back.

13 Refresh: FGR and Controller-Directed Per-Bank

13.1 Marking: MODIFIED — inherited mechanism, FGR density on top

`refresh_ctrl` is modified for a named, bounded change: DDR4's fine-granularity refresh adds a density dimension, and LPDDR4's per-bank refresh changes who picks the bank. Both changes sit on top of the mechanism scoria landed and verified — the retention interval, all-bank refresh, the JEDEC ±8 postpone/pull-in credit window, the `REFpb` machinery, and TASK-001's Modes A (elastic refresh), B (TCR) and C's sibling scheduling discipline — which andesite inherits as the policy base. The credit ceiling is untouched by every mode this block carries.

13.2 What scoria already has, and what andesite keeps

The inherited block carries: the tREFI interval counter; all-bank `REF` with JEDEC's ±8 postpone/pull-in scheduling; the elastic pull-in/postpone streaks of Mode A behind CSRs; Mode B's temperature-compensated tREFI derate; the bank rotor that mirrors the device's internal per-bank counter; and the `REF_STATS` telemetry. The formal evidence transfers with the mechanism: retention and accounting properties proven in `formal/scoria/refresh_ctrl`, with the per-bank retention arithmetic re-derived for LPDDR3 as scoria's book records.

One inherited subtlety rides with the block and deserves the same explicit passing-on scoria gave it: demand-aware pull-in must not treat CAM micro-gaps as idle. The sixteen-cycle (CSR-sweepable) idle confirmation exists because demand occupancy blinks off between bursts; releasing postponed refreshes into those gaps would fire pull-ins mid-stream. Copy the constant with its reason.

13.3 The DDR4 delta: FGR

DDR4's MR3 selects the refresh granularity — **1x, 2x or 4x** — trading refresh frequency against per-refresh length inside the same tREFI budget (JESD79-4). The controller's obligations:

1. **Hold the FGR select as a runtime CSR** (it programs MR3), so every density can be measured from one bitstream — the config-not-param rule again, and the only genuinely new surface on the DDR4 side.
2. **Scale the interval arithmetic by the FGR factor.** 2x refreshes twice as often at roughly half the length; 4x four times. The tREFI counter, the credit window's bookkeeping and the `REF_STATS` denominators all take the factor into account.
3. **Re-derive the retention property per factor.** The inherited proof covers the 1x arithmetic; each FGR factor changes the interval arithmetic the

proof checks. Carrying the 1x proof forward unchanged would prove the wrong thing while appearing green — the failure mode this family has been bitten by before, and the rule is the same one scoria applied to `REFpb`: a changed interval changes the proof, and the proof is never carried forward green.

4. **tRFC tracks the selected density.** The per-refresh busy window differs by granularity and is a runtime CSR like every other timing.

13.4 The LPDDR4 delta: the controller names the bank

This is the refresh change worth understanding, because it inverts the constraint scoria designed around. For LPDDR2/3, the `REFpb` command carries **no bank address** — the device's internal counter picks the bank, in a fixed round-robin, and scoria's rotor exists solely to *mirror* that counter so the bank timers know which bank is about to go busy. Occupancy-aware ordering was not available at any price; scoria's book struck the question as malformed.

JESD209-4 changes that: LPDDR4's per-bank refresh **does** carry the bank on the CA bus. The controller picks. Consequences:

1. **The rotor is replaced by explicit bank scheduling** on the LPDDR4 path — a real mechanism change, and the heart of why this block is MODIFIED.
2. **Sequential round-robins still work** (the rotor's old mirror image is one legal policy), so the DDR4-style default is available; but the scheduler now has the information to do better, and the policy hook belongs in the block.
3. **The advanced schemes become implementable** — out-of-order per-bank refresh, write-refresh parallelisation and the rest of the DARP family surveyed in andesite TASK-001. This book implements the commodity round-robin and leaves the survey's schemes where they are; the scheduling-policy planeing that scoria's TASK-001 built (CSR-select, reset-off, telemetry-instrumented) is exactly the hook they'd hang on.

13.5 Requirements

1. Offer all-bank `REF` at the MR3-selected FGR density for DDR4, with the JEDEC credit window (inherited), and per-bank refresh for LPDDR4 with controller-selected banks (new scheduling, inherited credit discipline).
2. All-bank versus per-bank, and the FGR factor, are **runtime CSRs**; every combination is measurable from one bitstream.

3. Retention is re-derived per FGR factor and for the per-bank arithmetic, in `formal/andesite/refresh_ctrl` when RTL exists; until then the obligation is recorded here and in Chapter 6.
4. A refresh of either kind is an ordinary occupant of the banks it touches: the bank timers own the banks, and the request/grant discipline with the scheduler is inherited — maintenance requests, never preempts.
5. `REFpb`-equivalent traffic must respect per-bank timing as any operation on that bank would.

13.6 Telemetry

`REF_STATS` and the Mode A/B instruments carry over, and the FGR factor plus the per-bank policy state join them — so a refresh sweep (density, policy, workload) is measurable in-system rather than inferred. A detector that has never fired is not evidence of anything; this block's telemetry exists so "refresh is fine" is a measurement, not a hope.

14 ODT: Dynamic, with a New odt_ctrl

14.1 Why static ODT doesn't survive DDR4

scoria holds ODT static: RTT_NOM is programmed in MR1 at init, the ODT pin is held at a fixed level through initialization, and nothing changes at runtime. That was defensible at DDR3-800 with one rank, and scoria's book says so plainly. It is insufficient for DDR4, for two reasons the standard builds in:

1. **RTT_WR.** DDR4 expects the termination to switch to a write-appropriate value *during write bursts* and back after them (JESD79-4). A static programming either over-terminates reads or under-terminates writes.
2. **RTT_PARK.** DDR4 adds a parked termination value (MR5) applied when no rank is selected — the idle state's impedance is a policy decision the standard exposes and boards exploit.

On top of the values, the **ODT pin itself is timed**: assertion and de-assertion follow the command stream with defined latencies (the ODTL family and the ODT turn-around timings, all named per JESD79-4 and all runtime CSRs). Termination on DDR4 is not a setting; it is a schedule.

14.2 What `odt_ctrl` owns

`odt_ctrl` is NEW. It is a policy block, not a datapath block — it touches no DQ bits; it decides what the DRAM's termination does and when, and it drives the ODT pin(s) accordingly.

Table 3.7: `odt_ctrl` responsibilities

Responsibility	Detail
Policy state	current RTT selection per rank: NOM / WR / PARK, CSR-programmed values at init, runtime-selectable policy
Write-burst coupling	on the scheduler's write grants, switch the writing rank's termination to RTT_WR for the burst and back after — the coupling is a command-stream tap, not a datapath tap
Pin timing	the ODTL assertion/de-assertion latencies and ODT turn-arounds as runtime CSRs, enforced on the pin
Rank coupling	multi-rank ODT (the rank not being

Responsibility	Detail
	accessed terminates per policy) — exercised when `NUM_RANKS > 1`, dormant at the single-rank design point but not special-cased in structure
Scheduler interface	ODT decisions ride the same request/grant channel as other maintenance-class traffic — odt_ctrl never blocks a command to make a termination point
Telemetry	ODT state history and transition counts, so a termination policy sweep is measurable

The init seam. During initialization the DRAM holds ODT high-impedance until CKE is registered active, then static — the init sequencer owns ODT through that window exactly as scoria's does, and hands ownership to `odt_ctrl` at the ready-for-operation boundary. Which block drives the pin in which state is a named, bounded seam the MAS pins.

LPDDR4. LPDDR4 has no ODT pin; its termination (DQ ODT) is programmed through the mode registers and is intentionally static for this edition — absorbed into `mode_register`'s LPDDR4 maps and the init sequence. The dynamic machinery above is DDR4-scoped, and the block's structure keeps that scoping explicit rather than dead-genericing over a pin LPDDR4 doesn't have.

14.3 Requirements

1. RTT_NOM, RTT_WR and RTT_PARK values are CSR-programmed at init from the board's decided values; the *policy* of when each applies is the block's logic.
2. Every ODT latency is a runtime CSR, named per JESD79-4, initialised from the design point's speed bin at CSR-derivation time (Ch 5).
3. ODT transitions never stall the command stream: the block consumes the schedule, it doesn't arbitrate it.
4. Telemetry distinguishes the policy states over time, so "dynamic ODT is on" is observable, and so is a policy that never leaves PARK.
5. Write CRC's exclusion (Ch 3.1) is recorded here too: CRC would couple to the write datapath and DM/DBI pins; its unblocking condition is named there.

15 The LPDDR4 Deltas, Chapter by Chapter

This book is DDR4-led with LPDDR4 carried per-chapter, the same one-book- both-memtypes shape scoria uses. This chapter collects LPDDR4's deltas as a reading aid — each entry points at the chapter that owns it, and none of them contradicts what that chapter says.

15.1 Where LPDDR4 differs

No bank groups. LPDDR4's 8 banks per channel stand ungrouped (JESD209-4). The long/short pair the scheduler learns for DDR4 degenerates to L = S, and the requirement on the arbiter and global timers is graceful degeneration, not a special case: same-group and cross-group checks collapse into one constraint per pair. Chapter 3.1.

Two x16 channels per package. LPDDR4 devices offer a pair of independent x16 channels. The design point exercises one channel (Chapter 2.4); the second channel is a build parameter, and the per-channel state that `ca_train_ifc` and the LPDDR4 refresh path already carry is why that parameter doesn't restructure anything. Chapter 2.4, 3.3.

The 6-bit DDR CA bus. LPDDR4 commands ride a 6-bit double-data-rate CA bus, two cycles per command, with a different encoding philosophy from DDR4's five-pin + address form. This is the formatter's NEW LPDDR4 CA submodule, kept beside — not merged into — the DDR4 path so each stays reviewable. Chapter 3.2.

MPC instead of ZQ opcodes. Calibration, training entry, and several feature modes ride the multipurpose command rather than dedicated opcodes. `zq_ctrl`'s NEW MPC submodule and `ca_train_ifc` share the issuer. Chapters 3.2, 3.3.

Controller-directed per-bank refresh. LPDDR4's per-bank refresh names the bank, replacing scoria's rotor with explicit bank scheduling and making the advanced schemes of andesite TASK-001's survey implementable. This edition implements the commodity round-robin. Chapter 3.4.

MR-programmed termination. No ODT pin; DQ ODT is programmed in the mode register set and held static this edition. The dynamic machinery of `odt_ctrl` is DDR4-scoped. Chapter 3.5.

DSM power states. Deep-sleep modes exist in the standard and are deferred here with the condition named in Chapter 3.1 — they wake the `powerdown_ctrl`/`dfi_signal_pack` pair and need a target that can measure power. Chapter 3.1.

Training. CA bus training and write-DQ calibration are LPDDR4-native requirements, carried by `ca_train_ifc` with firmware searches. Chapter 3.3.

Init. LPDDR4's initialization is its own ordered sequence — bus-carried reset, mode-register writes in JESD209-4's order, MPC ZQ calibration — with the same binding rule as DDR4's: the order is a citation, not a design choice. Chapter 3.2.

Inline CKE. LPDDR4 has no dedicated CKE pin; the CKE-equivalent state is encoded on the CA bus, so power-down and self-refresh entry are CA-bus transactions rather than a separate pin. The

formatter's `dfi_cke` output is DDR4-scoped; the LPDDR4 CA submodule handles CKE-equivalent states. Chapter 3.2 (MAS `ch02_blocks/01_cmd_formatter.md`).

15.2 What LPDDR4 does *not* change

Worth stating to keep the scope honest: the host side (nothing about LPDDR4 reaches the AXI4 front end), the datapath structure (the same read/write pipelines serve both memtypes; only the DBI pin handling is DDR4-side), the bank-timer and CAM machinery, and the evidentiary discipline. The family engine covers both memtypes because the deltas are concentrated where they should be: the command encoding, init/training, refresh policy, and termination.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

16 DFI v4.0 Master Interface, and the v3.1 Delta

16.1 The revision decision

andesite targets **DFI 4.0** (§1.0 Overview, §2.2.3 DFI Signals) — the DDR4-era revision per the bootstrap spec §2. The reasoning is short and on the record: scoria's "deliberately unimplemented" table is almost entirely the DDR4/LPDDR4 surface of its v3.1 — `dfi_act_n*`, `dfi_bg*`, CA parity and `dfi_alert_n*`, DBI, CA training. andesite's job list *is* that table. Targeting v3.1 and hand-adding the DDR4 pins would rebuild the boundary scoria already drew; targeting 4.0 makes the boundary match the work.

Citation discipline for this chapter. The DFI 4.0 specification is now on disk (`/mnt/data/github/dfi-specs/DDR_PHY_Interface_Specification_v4_0.pdf`) and was studied on 2026-10-04 as andesite TASK-005. Clause references below are real DFI 4.0 section numbers; any remaining `§TBC(TASK-005)` marks a claim the study could not confirm. Signal *names* are public and stated plainly.

16.2 What does NOT change

The datapath is inherited entirely. One async-FIFO CDC, bubble-free, the unit of transfer a DFI word, at the inherited 1:4 frequency ratio. The phase-multiplied buses carry: the `_pN` generalization scoria adopted from v3.1 still stands, and the 1:4 ratio remains a defined mode of the newer revision (§4.9 Frequency Ratios Across the DFI).

The init and error surface carries. `dfi_init_start` / `dfi_init_complete`, `dfi_error` and `dfi_error_info` are scoria's already (§3.5.1 Initialization, §3.8 Error Interface); the DDR4 CA-parity detail that `dfi_error_info` was widened to carry is now used rather than ignored.

`dfi_reset_n` is inherited. scoria drives it from the init sequencer and presents it at the DFI boundary under its DFI name; andesite keeps that convention (§3.1 Control Interface, Table 3; Ch 3.2).

The low-power requests stay exposed and unacknowledged-by-default. `dfi_lp_ctrl_req` / `dfi_lp_data_req` remain presented, with the inherited rule that an unacknowledged request times out and reports — never blocks. The dormant-pair condition of Chapter 3.1 governs whether anything ever consumes them (§3.7 Low Power Control Interface).

16.3 What changes and matters: the 3.1 → 4.0 delta

Table 4.1: The 3.1 → 4.0 delta — what scoria left unimplemented is andesite's work list

Signal / behavior (DFI 4.0)	What scoria drove at v3.1	What andesite drives	Citation
`dfi_act_n*` — the	absent; ACT was	driven from	§3.1 Control

Signal / behavior (DFI 4.0)	What scoria drove at v3.1	What andesite drives	Citation
fifth command pin	an encoded command	`dfi_cmd_formatter` (MODIFIED); ACT_n plus RAS/CAS/WE forms the activate	Interface
`dfi_bg*` — bank-group address	absent; no bank groups	driven with BG0/BG1 during activate, from the `addr_mapper` decode	§3.1 Control Interface
`dfi_cid*` — chip ID	absent	not driven at `NUM_RANKS = 1`; presented constant, like scoria's data-phase chip selects	§3.1 Control Interface, §4.3 3DS Stack Support
`dfi_geardown_en` — gear-down mode	n/a — not in v3.1	`init_sequencer` drives entry after init; the CA bus then runs at half rate (§3.13, Ch 3.2)	§3.13 Geardown Mode
`dfi_parity_in` — CA parity	`dfi_error`/ `dfi_error_info` carried the indication; parity encodings ignored	`dfi_cmd_formatter` generates parity; the counting machinery's hw/fw split is an open question (Ch 6)	§3.5.7 CRC and CA Parity
`dfi_alert_n*` — parity/training alert	absent	input to a NEW `alert_n` handling submodule at the boundary; surfaces through `dfi_error`	§3.5.7 CRC and CA Parity

Signal / behavior (DFI 4.0)	What scoria drove at v3.1	What andesite drives	Citation
`dfi_wrdata_mask` (write DBI) / `dfi_rddata_dbi` — Data Bus Inversion	absent	write DBI reuses `dfi_wrdata_mask` when DBI is enabled; read DBI is `dfi_rddata_dbi`; MR5-programmed through the MODIFIED aligner and serializer	§3.2.1 Write Data Mask/Write DBI, §3.3.1 Read DBI, §4.4 Data Bus Inversion
`dfi_phy_rdlvl_cs`, `dfi_lvl_pattern`, `dfi_lvl_periodic` — read leveling and pattern	present in v3.1 as `dfi_phy_rdlvl_cs_n`, unused (write leveling only)	driven by `rdlvl_ifc` for MPR-based read leveling (Ch 3.3); v4.0 drops the `_n` suffix on training chip selects	§3.6 DFI Training Interface, §3.6.1 Read Training Operation, §3.6.6 dfi_lvl_pattern Encoding, §3.6.7 Periodic Training Flag
`dfi_calvl_*`, `dfi_calvl_capture`, `dfi_phy_calvl_cs` — CA training	absent	driven by `ca_train_ifc` for LPDDR4 CA/WDQ training (Ch 3.3, 3.6); v4.0 drops the `_n` suffix on `dfi_phy_calvl_cs` and adds LPDDR4 VREF signals (`dfi_calvl_data/done/result/strobe`)	§3.6.4 CA Training Operation, §4.12.7 CA Training
frequency-ratio options beyond 1:4	n/a at this design point	not exercised; deferred until a board PHY requires one	§4.9 Frequency Ratios Across the DFI

Study correction (2026-10-04, andesite TASK-005). The v3.1 per-CS leveling handshake `dfi_phylvl_req_cs_n` / `dfi_phylvl_ack_cs_n` does *not* transfer unchanged into DFI 4.0. The 4.0 spec removes those PHY-requested training signals and replaces them with PHY Independent Mode (§3.6.8 PHY-Initiated Training in DFI Training Mode). All training chip-select signals are also renamed from `*_cs_n` to `*_cs` in 4.0 (e.g. `dfi_phy_rdlvl_cs_n` → `dfi_phy_rdlvl_cs`, `dfi_phy_calvl_cs_n` → `dfi_phy_calvl_cs`). andesite's training interface must use the v4.0 names; any reuse of v3.1-style `phylvl_*` handshakes is an andesite-specific convenience, not a DFI 4.0 signal.

A second correction: there is no `dfi_wrdata_dbi` pin in DFI 4.0. Write DBI is carried on the existing `dfi_wrdata_mask` bus when DBI is enabled and `phydbi_mode = 0` (§3.2.1). Read DBI has the dedicated `dfi_rddata_dbi` signal (§3.3.1).

The trap to record, carried from scoria's chapter. A naive signal-name diff between revisions reports phase-indexed names as appearing and disappearing with each revision's notation. They don't; the `_pN` form is a generalization, and a 1:4 implementation keeps four concrete phases. An implementer who trusts the diff rewrites a working, verified datapath for no reason — so the trap stays on the record here, one revision later.

16.4 What andesite does NOT implement

Named, so the absence is a decision:

Table 4.2: Deliberately unimplemented at the 4.0 boundary

Group	Why not
The DDR5-oriented features the DFI 4.x revision family also carries (ch00's not-in-scope line)	No DDR5 in this controller; named here so the boundary is a decision (§2.2.1 Global Features, §2.2.2 Memory Topology-Specific Features)
Frequency ratios beyond the inherited 1:4	No board PHY to consume them; the datapath's phase structure transfers at the gear it was verified at (§4.9 Frequency Ratios Across the DFI)
`dfi_lp_*` consumption	The dormant-pair condition of Ch 3.1; the requests stay presented per the timeout rule above (§3.7 Low Power Control Interface, §4.13 Low Power Control Handshaking)

Targeting the newer revision does not mean implementing all of it — the same sentence scoria's chapter closed on, one revision later.

16.5 The MAS seam

This chapter names the boundary; the MAS ch03 pins it — every signal above appears there as a pin-level row (name, direction, clock, reset value, source block), and the clause column there carries the same DFI 4.0 section references. The DV framework's DFI signal catalog — the check that, for scoria, caught a missing required signal by name — runs against this table when the BFM exists (andesite TASK-005).

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

17 AXI4 Slave and APB CSR (both inherited)

17.1 The host side is unchanged in shape

Everything on the host side of the controller transfers from scoria without structural change: the AXI4 slave, the width-gearing converters that let the host bus width differ from the DRAM's (x8 DDR4, x16-per-channel LPDDR4 at this design point), the APB slave into the register block, and the in-order commit discipline. Nothing about DDR4 or LPDDR4 reaches this side of the controller, and the front-end blocks that serve it are the green set of Figure 2.1.

The AXI4 shape is inherited as a *shape*, not just a protocol: the same ID handling, the same burst chopping against the DRAM's page, the same read-reorder and write-data CAMs. scoria's book is the specification for those blocks and andesite's MAS references it rather than rewriting it.

17.2 The register block

The CSR block regenerates from an `andesite_csr.rdl` through the same PeakRDL flow scoria uses. Two rules carry, both load-bearing:

1. **Registers are named, not offset-numbered, in books.** Offsets live in the generated documentation, not in prose; a register's name is its handle everywhere in this book, the MAS, and the kmap book. The house peakrdl wrapper generates the offset map when the RDL lands.
2. **The generation flow is inherited; the contents grow.** Every new timing (the long/short pairs, the ODT latencies, the FGR factor), every new policy select (placement, elastic refresh carried forward, DBI enables), and the training interfaces' telemetry join the register map. Chapter 5 owns the inventory; the growth is content, not mechanism.

17.3 The seam with the rest of the book

The host side's only contacts with this book's deltas are telemetry (the training and ODT instruments are host-visible, per the four-state rule of Chapter 3.3) and the width gearing's view of the LPDDR4 channel width. Both are CSR-visible; neither is structural.

18 andesite_pkg, and Build-Time vs Runtime

18.1 The package

andesite carries its own `andesite_pkg` from day one. The shared-core design — the family memtype enum, the shared timing-struct inventory, the migration plan with its conditions — is owned by the family doc 01_mem_ctrl_pkg.md and is **referenced here, not restated**. What belongs to this chapter is only what andesite's package owns.

The memtype enum note. andesite has no legacy CSR to disturb — the shipping-design constraint that kept pumice and scoria at one bit does not apply here. `andesite_pkg` therefore takes the family design as-is: one LP axis bit plus the two-bit generation field, values assigned in the family doc. The day-one note that matters for the register block: the memtype CSR field in `andesite_csr.rdl` is three bits from birth, and the `PHY_TIMING` shadow carries it without a width migration. When the family migration lands (both shipping controllers moving together, per the family doc's conditions), `andesite_pkg` folds into `mem_ctrl_pkg` with only an import change.

The opcode set. scoria's 4-bit internal `OP_*` encoding carries (`scoria_pkg.sv:47-60` — NOP through the refresh and ZQ ops), and this tier adds one member: `OP_MPC`, the LPDDR4 multipurpose command whose opcode encoding on the 6-bit CA bus the formatter's new submodule owns. Gear-down is a sequenced mode entry, not an opcode; the long/short timing pairs are values, not opcodes. Nothing else in the command set changes shape.

18.2 The rule: timings are runtime, geometry is build-time

Inherited from the family doctrine and restated because it is the single most load-bearing convention here:

Every enforced timing is a runtime CSR. Not a parameter. A timing compiled in cannot be swept, and a controller whose timings cannot be swept cannot be characterized. This edition's additions — the tCCD_L/S and tRRD_L/S pairs above all — follow the rule absolutely: they are the feature DDR4 adds, and they are runtime like everything else.

Corollary — a runtime timing CSR is a count of controller-clock cycles, so its value is bound to the MC clock frequency. "Runtime" does not mean "frequency-independent": the register holds cycles, not nanoseconds, and the correct count changes when the clock does. A frequency change invalidates every cycle-counted CSR at once — the Table 5.1 timing rows and every new group in Table 5.2 — and software must re-derive the whole set from the JESD ns/nCK values before traffic resumes. Stale values violate real DRAM timings and present as data corruption, not errors.

Build-time is reserved for what cannot be runtime: bus widths, the DFI gear ratio, CAM depths, bank and rank counts, and the geometry below. The test is whether changing it changes the amount of hardware.

The design point of Chapter 2.4 fixes the geometry, and these two rows are the book's canonical statement of it — repeated here byte-identical so a reader of either chapter sees the same string:

Table 5.0: The design-point geometry, byte-identical to Chapter 2.4

Side	Geometry	Source
DRAM, DDR4 side	DDR4-1600 x8, 4 bank groups × 4 banks = 16 banks, MT40A1G8-class	bootstrap spec §2 design point
DRAM, LPDDR4 side	LPDDR4-1600 x16, 8 banks per channel — ungrouped; the package offers two x16 channels	bootstrap spec §2; JESD209-4

Table 5.1: Build-time versus runtime

Kind	Build-time or runtime	Examples
Bus and geometry	build	data width, address width, the Table 5.0 geometry, `NUM_RANKS = 1`, CAM depths
DFI gearing	build	the 1:4 frequency ratio, which must equal the PHY's phase count
Memtype	build	the family `memtype_e`, three bits
JEDEC timings	**runtime**	every command-spacing and recovery parameter
New DDR4 timings	**runtime**	tCCD_L/S, tRRD_L/S, the FGR-scaled tRFC family, the ODT latency family
LPDDR4 timings	**runtime**	the JESD209-4 command and training intervals
Policy selects	**runtime**	page policy, refresh policy (FGR factor, per-bank policy), ZQCS

Kind	Build-time or runtime	Examples
		placement (inherited), ODT policy, DBI enables, gear-down select, CA-parity scope, and the carried-forward TASK-001 mode selects

Important — the frequency ratio must equal the PHY's phase count, set identically on both sides. Inherited from scoria, not negotiable: a mismatch is a functional break that presents as data corruption, not as an error.

18.3 New CSR groups

The register set grows past scoria's. Exact offsets are not fixed in this edition — they come from the RDL, authored with the RTL, and registers are named, not offset-numbered, everywhere in the books (Ch 4.2). The content is specified:

Table 5.2: New CSR groups

Group	Contents
Init	the `tINIT*` family, `tDLLK`, `tZQinit`, `tMRD`, `tMOD` — named per JESD79-4 / JESD209-4, values recorded as open question Q1 until read from cold storage
Refresh	the FGR select (MR3 image), the per-bank refresh policy, and the inherited `REF_CTRL` elastic/TCR fields with their `REF_STATS` telemetry
ODT	RTT_NOM / RTT_WR / RTT_PARK programmed values, the ODT latency family, the policy select, and ODT telemetry
Training	the inherited write-leveling windows, the read-leveling (MPR) windows, the LPDDR4 CA/WDQ windows, per-chip-select results, and the four-state telemetry of Ch 3.3

Group	Contents
DBI / parity	read/write DBI enables (MR5 image), CA-parity enable and scope
LPDDR4	channel configuration, MPC calibration interval, CA-training control

Notes that carry. `tWLMRD`'s open maximum stays a controller-defined CSR timeout with a distinct status bit — the family rule, quoted in Ch 3.3. The gear-down select programs a mode the datapath never sees; it exists so one bitstream can run both CA rates. And the FGR select's CSR exists so the retention re-derivation Ch 3.4 requires can be *swept*, not just asserted.

18.4 The duplication, recorded again

`pumice_pkg`, `scoria_pkg` and `andesite_pkg` will carry near-identical type definitions for a period. That is deliberate and time-boxed, owned by the family doc's migration plan, and recorded here — as it was recorded in scoria's Chapter 5 — so a later reader does not "fix" it by widening a shipping CSR. That widening is precisely the thing the arrangement exists to prevent.

19 Verification Strategy, and the Open Questions

19.1 The target is the DFI boundary, in simulation only

Verification is against a DFI v4.0 bus functional model, in simulation. No board is named for this controller — the 7-series targets this repo's boards carry do not support DDR4 — and that is a deliberate strengthening of the rule, not a weakening: with no board even conceivable, nothing can be left for bring-up to find.

The in-house BFM is DFI 4.0-partial, not absent. The DV repository's CocoTBFramework DFI component exists at `/mnt/data/github/RTLDesignSherpa-DV/src/CocoTBFramework/components/dfi/` and already carries a CA-parity behavior, the v4.0 behavior class (`behaviors/v4_0.py`), and a complete v4.0 signal catalog (`dfi_signal_catalog.py`). What it lacks for the andesite design point is enumerated below. Acquisition is therefore not the problem — gap analysis and extension are, and that is andesite TASK-005 (amended 2026-10-04, when the DFI v4.0 spec PDF also surfaced in the operator's research storage at `/mnt/data/github/dfi-specs/`). The DFI 4.0 clause citations in this book and the MAS were confirmed or corrected on 2026-10-04; any remaining `§TBC(TASK-005)` marks a claim the study could not confirm. The BFM is configured to the design-point geometry of Chapter 2.4 — the bank-group geometry on the DDR4 side, eight ungrouped banks per channel on the LPDDR4 side, x8 / x16 widths, four DFI phases — because verifying at the geometry the design point uses is free, and verifying at a different one silently weakens every result.

The verification target is therefore three counterparties, not one. The in-house DFI 4.0 BFM is the primary checker: it owns the protocol and the pin-level contract. DRAMsim3, cited in both the DDR4 and LPDDR4 research indexes (Li et al. 2020), is the cycle-accurate second reference for the command stream — it cross-checks that the sequence the controller issues is legal and produces the same DRAM state. For LPDDR4, the missing `lpddr4_ca_map.py` encoder (BFM gap G1) is closed by andesite TASK-010, and the BFM round-trip encode/decode tests for the CA encoder become the third counterparty — they prove the MAS CA tables and the RTL encoder agree before DRAMsim3 ever sees the stream.

BFM gap list vs HAS Table 4.1 / andesite design point. CLOSED 2026-10-04 by RTLDesignSherpa-DV commit `61a27d8fee3b` (`dfi(bfm): close DFI 4.0 BFM gaps TASK-010`; +30 tests, 1538 unit passing). Known omissions recorded there: no LPDDR4-1600 CSV (JESD209-4E publishes no fixed speed-bin table — timings are density/vendor-dependent), DES/PDE idle entries omitted from the CA map, ACT modeled as one 4-edge command. Gaps were against the files read for this study: `behaviors/base.py`, `v3_1.py`, `v4_0.py`, `v5_2.py`, `v6_0.py`, `registry.py`; `ca_map.py`, `ca_transport.py`, `ddr5_ca_map.py`, `dfi_signal_types.py`; `jedec/README.md` and the CSV list in `jedec/`.

#	Gap	Evidence	Impact on first andesite sims

G1	**LPDDR4 6-bit CA command map.** No `lpddr4_ca_map.py` exists; `ca_map.py` ships only HBM4 and DDR5 maps. The `lpddr_ca.py` encoder is LPDDR2-only, not LPDDR2/3 as the task amendment assumed.	`ca_map.py` contains `HBM4_ROW_CA_MAP` and `HBM4_COL_CA_MAP`; `lpddr_ca.py` docstring and body encode only JESD209-2F Table 60 (LPDDR2). No `lpddr3_ca.py` or `lpddr4_ca_map.py` present.	Blocks item 6 (LPDDR4 CA bus checker) until added. The kmap-generated tables in `docs/kmaps/generated/` are the source of truth for the encoder.
G2	**Design-point JEDEC timing CSVs missing.** The andesite design point is DDR4-1600 / LPDDR4-1600. Vendored CSVs exist for `ddr4-2400.csv`, `ddr4-2666.csv`, `ddr4-3200.csv`, but no `ddr4-1600.csv`; no LPDDR4 CSV of any speed exists.	`jedec/` file listing; README §“Devices with no vendored profile”.	The DRAM-state timing checker cannot run at the design point without new CSVs or a `timings_from_params` call.
G3	**Gear-down mode behavior.** `dfi_geardown_en` is in the signal catalog, but `DFIv4_0Behavior` has no method to sample or validate the geardown	`dfi_signal_catalog.py` lists `geardown_en`; `behaviors/v4_0.py` has no geardown method.	Item 2 (gear-down mode switch) can be signal-level checked, but protocol-aware BFM validation of entry/exit is missing.

	entry/exit handshake.		
G4	**LPDDR4 CA VREF training specifics.** `DFIv4_0Behavior.training_step()` detects CA training via `calvl_en`/`calvl_req` inherited from v3.1, but does not model `dfi_calvl_data`/`done`/`result`/`strobe` or the VREF-sweep sequence.	`behaviors/v4_0.py` `training_step`; `dfi_signal_catalog.py` lists the LPDDR4 VREF signals.	CA/WDQ training (item 7) can be handshake-checked, but VREF-sweep BFM feedback is not available.
G5	**Per-slice read leveling.** `DFIv4_0Behavior` reports `slice_idx=0` for all training events; v4.0 makes `rdlvl_req`/`en` per-slice.	`behaviors/base.py` and `v4_0.py` return `TrainingEvent(slice_idx=0)`.	Item 7 training protocol checks are not slice-aware.

Integration note. The BFM path is `/mnt/data/github/RTLDesignSherpa-DV/src/CocoTBFramework/components/dfi/`. Version files read: `behaviors/base.py` (v2.1 baseline), `behaviors/v3_1.py`, `behaviors/v4_0.py`, `behaviors/v5_2.py`, `behaviors/v6_0.py`, and `behaviors/registry.py`. The andesite BFM instance should select `DFIVersion.V4_0` via `behavior_for(V4_0)`. The signal catalog already validates the HAS Table 4.1 / MAS Table 3.1 signal names once the `_cs_n` → `_cs` v4.0 renames are applied; the gaps above are behavioral / encoding extensions, not missing DFI 4.0 signals.

The boundary is chosen for the reason pumice chose it: it is the one interface where the controller's obligations are fully specified by a standard, so a model can be authoritative rather

than approximate. pumice's experience — BFM-passing code passes on hardware modulo PHY training — is exactly the residue this family pushes into firmware.

19.2 The inherited-bring-tests argument

The inherited blocks bring their verification with them. scoria's suites — 221 tests and 9 formal blocks at last measure — transfer with the blocks they test, and pumice's board measurements sit upstream of those. The obligation this controller adds is not to re-prove the front end; it is to prove the deltas. That argument is load-bearing: it is why the marking table is also the test-plan structure.

19.3 What must be verified, beyond the inherited suites

Named now, because "new coverage" that isn't enumerated doesn't happen. Every item below is ultimately checked against at least one of the three counterparties named above — the DFI 4.0 BFM, DRAMsim3, or the LPDDR4 CA encoder round-trip tests:

1. **The init sequences, in JEDEC's order, both memtypes.** A checker that asserts the order — RESET# held, the CKE waits, the MR3-MR6-MR5-MR4-MR2- MR1-MR0 order on the DDR4 side and JESD209-4's order on the LPDDR4 side, ZQ closing init — because order is what this family has historically gotten wrong (pumice's EMRS3-first sequence, benign but wrong).
2. **Gear-down as a mode switch.** Entry programmed, bus rate switched on both sides of the DFI boundary, and full-rate init still proven with gear-down disabled. The handshake is §3.13 Geardown Mode / §4.18 Use of the Geardown Mode; the *behaviour* is verifiable the moment the BFM supports it (see G3 above).
3. **CA parity and `alert_n` as a protocol.** The counting machinery exercised, an injected parity fault raising `alert_n`, and the error surfacing through `dfi_error` — including the firmware-assist split if Q4 below lands there.
4. **ODT as a schedule.** The `odt_ctrl` policy states over time: RTT_WR confined to write bursts, PARK in the idle policy, the latency family enforced on the pin, and a trace that shows a policy that never leaves PARK (which is what "dynamic ODT off" looks like from outside).
5. **FGR retention, re-derived per factor.** The inherited proof covers 1x arithmetic; 2x and 4x change the interval arithmetic. Each factor gets its own property in `formal/andesite/refresh_ctrl`; none is carried forward

green. Same rule for the LPDDR4 per-bank arithmetic with controller-named banks.

6. **The LPDDR4 CA bus as a two-cycle encoding.** Command-by-command checker against the truth table the kmap book pins — the tables in `docs/kmaps/generated/` are the checker's expected values, which is what the generated kmap book is *for*.
7. **Training as protocol, timeouts exercised deliberately.** Write leveling (inherited), read leveling, and CA/WDQ training: handshakes, windows, and every timeout path — a timeout that has never fired is an untested path on the only path that reports failure.
8. **The dormant pair stays on the record.** `powerdown_ctrl` and `dfi_signal_pack` verified uninstantiated-in-tree, so the dormant disposition of Chapter 3.1 can't silently rot into dead code or silently resurrect.

The formal area is the right home for the spacing properties. No assertions go in the RTL — the family rule, tool-compatibility, quoted in Chapter 1.2. `cmd_history_checker` grows DDR4's long/short parameters before the RTL is trusted, exactly as scoria grew DDR3's; the spacing properties that checker shadows — tCCD_L/S, tRRD_L/S, the ODT turn-arounds — are the first candidates for `formal/andesite`.

19.4 Status, 2026-10-04

No andesite RTL exists. This book is a v0.1 written from the delta analysis, the same posture scoria's v0.1 had — and the evidentiary rule exists because that posture can specify something unbuildable. What exists now: this book, the vault lane, and the family docs. What is measured: the *predecessor's* evidence, cited chapter by chapter where it transfers.

19.5 Open questions

Table 6.1: The open questions, each with the condition that answers it

#	Question	Named condition
Q1	The numeric init constants (`tINIT*`, `tDLLK`, `tZQinit`, `tMRD`, `tMOD`)	Read from JESD79-4 / JESD209-4 in cold storage at CSR-derivation time (first RTL pass). Named, not invented, per the evidentiary rule
Q2	Write CRC: un-defer?	A characterization or

#	Question	Named condition
		board campaign that needs end-to-end write protection; the addition is bounded to the write datapath (Ch 3.1, 3.5)
Q3	Gear-down coverage scope in the first sims	**ANSWERED 2026-10-04.** The BFM gained gear-down protocol validation (G3, closed in DV commit 61a27d8fee3b), so the first sims can protocol-check the mode switch itself; the full-rate-only fallback is no longer needed
Q4	CA parity: hardware counter vs firmware assist	A scope decision at MAS/RTL time; either way the protocol check of item 3 above must pass, and the split is recorded in the MAS
Q5	LPDDR4 DVFS/DSM, and the dormant pair's waking	A low-power consumer and a target that can measure power; wakes `powerdown_ctrl`/`dfi_signal_pack` per Ch 3.1
Q6	DFI 4.0 clause confirmation and BFM provenance	**Closed on 2026-10-04.** The spec is acquired, every `§TBC(TASK-005)` suffix in this book and the MAS is confirmed or corrected, and the in-house BFM integration note and gap list are above. G1-G5 remain as

#	Question	Named condition
		extension work, not blockers to clause confirmation.
Q7	PB-REF evaluation (per-bank-group refresh accounting)	**DEFER WITH NAMED CONDITION.** PB-REF is a bounded MODIFIED-plane addition to `refresh_ctrl` behind the CSR policy hook scoria's TASK-001 built. Un-defer when RTL bring-up characterization shows refresh stalls attributable to device-wide `tRFC` serialization. Bhati 2016's FGR/per-bank findings (Table 2) give the cost/benefit summary: 2x/4x FGR shortens `tRFC` by refreshing fewer rows per command but issues refreshes more often, while per-bank refresh lets other banks stay available. PB-REF is orthogonal to the landed FGR work — FGR is a density/granularity choice, PB-REF is a bank-group stagger pattern — so either can land without the other.

19.6 What would make this a 1.0

Three things, in order — the same three scoria's book named, one generation later:

1. **RTL exists, and this document is reconciled against it block by block.** Every INHERITED marking confirmed or corrected, every MODIFIED change verified bounded as named, every NEW block found where this book said it would be. A marking that turns out wrong is a defect in this document, and the correction goes in the revision history rather than being dropped silently.
2. **The exact CSR map, from the RDL.** The map cannot honestly precede the RDL; Chapter 5's groups become offsets when `andesite_csr.rdl` lands.
3. **andesite TASK-005 closed.** The DFI 4.0 clauses confirmed and the BFM selected — every `§TBC(TASK-005)` suffix in this book is either a citation or a correction, and the verification strategy above has a counterparty with the gap list in the BFM paragraph.

Until then this is a 0.1: a specification that knows what it doesn't know, and says so on the record.